

VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
School of Computer Science and Engineering



**DEVELOPMENT OF AN INTELLIGENT WEB-BASED
FRAMEWORK FOR EARLY DETECTION OF
BURNOUT IN THE INDUSTRIAL WORKFORCE**

*A Microservice Approach Using the Copenhagen Burnout Inventory
and Random Forest Regression*

By

Nguyen Phuc Dat

Student ID: ITITIU22028

Instructor

Dang Tam Nhan

A thesis submitted to the School of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Science

Ho Chi Minh City, Vietnam

2026

**DEVELOPMENT OF AN INTELLIGENT WEB-BASED FRAMEWORK FOR
EARLY DETECTION OF BURNOUT IN THE INDUSTRIAL WORKFORCE**

APPROVED BY:

Dang Tam Nhan, Supervisor

Committee Member

Committee Member

THESIS COMMITTEE

ACKNOWLEDGMENTS

First of all, I want to thank my supervisor, Dang Tam Nhan for guiding me and giving me precious advice and encouragement in writing this thesis. I wish to thank International University – Vietnam National University Ho Chi Minh City for giving me a very nurturing academic environment, my classmates and learned and experienced instructors. Then I want to mention my classmates Tran Luu Hong Phuong and Luong Quang Huy who have been standing side by side with me during this period. Learning and studying together, doing group study, as well as participating in the research that we have been allowed to join, helped me build a better thesis than what you are reading now. Also, their trust in my abilities is one of the greatest motivations that pushed me to where I am today. Lastly, I want to thank my family for their wholehearted support, material and emotional, they have provided me. Thank them for giving me everything I need to pursue my education. That is one of the greatest things they have ever done for me.

Nguyen Phuc Dat

Ho Chi Minh City, May 7, 2026

Contents

1	Introduction	1
1.1	Background	1
1.2	Motivating Scenario and Use Cases	1
1.3	Problem Statement	2
1.4	Scope and Objectives	2
1.5	Research Questions	3
1.6	Assumptions and Proposed Solution	4
1.7	Thesis Structure	5
2	Literature Review	7
2.1	Occupational Burnout	7
2.1.1	Definition and Impact on the Industrial Workforce	7
2.1.2	Burnout Measurement Instruments	8
2.1.3	The Copenhagen Burnout Inventory in This Thesis	8
2.2	Machine Learning for Health Prediction	9
2.2.1	Supervised Regression	9
2.2.2	Linear Regression	9
2.2.3	Random Forest Regression	10
2.2.4	Model Evaluation Metrics	11
2.3	Web Application Architectures	12
2.3.1	Microservice vs Monolithic Architecture and why I choose Mi- croservices	12
2.3.2	RESTful API Design	12
2.4	Related Technologies	13

2.4.1	React and Next.js for Frontend Development	13
2.4.2	Node.js and Express for Backend Services	14
2.4.3	FastAPI for Machine Learning Serving	14
2.5	Related Work	15
3	Methodology and System Design	17
3.1	System Architecture Overview	17
3.2	Database Design	18
3.2.1	MySQL Schema	18
3.2.2	MongoDB Collections	20
3.3	Backend Design	21
3.3.1	Layered Architecture	21
3.3.2	Authentication and Authorisation	22
3.3.3	Quiz Scoring Pipeline	23
3.3.4	Alert Rules	24
3.4	AI Microservice Design	24
3.4.1	Dataset Description	24
3.4.2	Feature Selection Justification	25
3.4.3	Data Preprocessing	25
3.4.4	Model Selection	26
3.4.5	FastAPI Service Architecture	27
3.5	Frontend Design	28
3.5.1	Technology Stack and Design Decisions	28
3.5.2	Route Structure and Navigation	28
3.5.3	Design System	30

3.5.4	Data Flow Pattern	30
4	Implementation and Results	32
4.1	Development Environment	32
4.2	Backend Implementation	32
4.2.1	Project Structure and Configuration	32
4.2.2	Authentication Flow Implementation	33
4.2.3	Quiz Scoring Implementation	34
4.2.4	Dashboard Aggregation	34
4.2.5	Alert System Implementation	35
4.2.6	API Endpoint Summary	35
4.3	AI Service Implementation	36
4.3.1	Model Training Results	36
4.3.2	FastAPI Prediction Endpoint	39
4.3.3	Integration with Express Backend	40
4.4	Frontend Implementation	40
4.4.1	Authentication Pages	40
4.4.2	Employee Portal	41
4.4.3	Admin Portal	45
4.4.4	Reusable UI Components	50
4.4.5	State Management with TanStack Query	51
5	Discussion and Evaluation	52
5.1	Model Performance Analysis	52
5.2	Functional Testing	53
5.3	System Feature Evaluation	55

5.4	Answering the Research Questions	56
5.5	Comparison with Related Work	57
6	Conclusion and Future Work	59
6.1	Conclusion	59
6.2	Contributions	60
6.3	Achievement of Objectives	60
6.4	Limitations	61
6.5	Future Work	62
	List of References	64
	Appendices	68
	Appendix A: Complete API Reference	68
	Appendix B: Database Schema	71
	Appendix C: CBI Questionnaire	77

List of Tables

1	Comparison with related work.	16
2	Dataset columns used for model training.	24
3	Primary design tokens.	30
4	Development tools and versions.	32
5	Summary of backend API endpoints.	36
6	Model performance comparison on the test set.	37
7	Functional test cases and results.	53
8	Evaluation of system objectives.	55
9	Complete API endpoint reference.	68

List of Figures

1	Structure of the thesis.	6
2	High-level system architecture.	17
3	Entity-relationship diagram of the MySQL database.	18
4	Layered architecture of the Express backend.	21
5	Quiz submission scoring pipeline.	23
6	Architecture of the Python/FastAPI AI microservice.	27
7	Application route structure and navigation flow.	29
8	Random Forest feature importance scores.	37
9	Distribution of Burn Rate values in the training dataset.	38
10	Pearson correlation heatmap of features and Burn Rate.	38
11	Actual vs. predicted burn rate for both models.	39
12	Residual analysis for the Random Forest model.	39
13	Login page with split-screen layout.	41
14	Registration page.	41
15	Employee dashboard with burn rate gauge and trend chart.	42
16	CBI assessment page — Step 1: Personal Burnout.	43
17	Assessment results page with score cards and support resources.	44
18	Quiz submission history with expandable response detail.	45
19	Admin command centre dashboard.	46
20	Department detail — Employees tab with filters and sorting.	47
21	Department detail — Analytics tab with burnout and KPI visualisations.	47
22	Admin employee detail with burnout and KPI analysis.	48
23	Admin alerts page with filtering.	49

24	Help resource management with CRUD operations.	50
----	--	----

ABSTRACT

Workers' mental health is one of the most pressing problems requiring attention at this stage of the world. Following the COVID-19 pandemic, workers are returning to their jobs with increased psychological instability, and turnover rates are rising. The workload, shift schedules, and repetitive tasks can exacerbate pre-existing mental health issues or lead to mental exhaustion in those who were previously mentally healthy. Previously, methods were developed for diagnosis rather than prevention, meaning that workers already burned out and symptoms had worsened, requiring significant resources and time for recovery or treatment, leading to reduced productivity and work quality.

This thesis was conceived to design and implement an intelligent web-based framework capable of early prediction of burnout rates in industrial work environments where employees work heavy workloads across multiple shifts. The system utilizes Copenhagen Burnout Inventory (CBI), a researched and validated burnout measurement tool. Along with test data, the system utilizes a Random Forest regression model (trained with a dataset of approximately 22,000 rows) to predict and provide burnout rates for test-takers (employees). Simultaneously, it reports and updates this information securely and privately to administrators (HR staff or other management levels), allowing companies and individuals at risk of burnout sufficient time to react.

This framework is based on a microservices architecture with three main parts: a frontend with NextJS (React), a backend with ExpressJS (NodeJS), and an AI service with FastAPI (Python). I use MySQL for relational databases with clear structures and relationships data, such as employee personal information, work performance, department information, job titles, etc., and MongoDB for document data, such as CBI test questions. The frontend has two main user flows: for employees and for

administrators. Employees can view their own information, take CBI tests, and access resources to support their mental health; administrators can view dashboards to see overall company or department results, receive alerts from employees with high burnout rates, and customize supporting resources. The backend collects the results sent from the frontend, processes and calculates them before saving them to the database or sending them to the AI service for calculation. The AI service will include a model trained with datasets, calculation formulas, and APIs for the backend.

The model in this thesis was trained with the Random Forest regression algorithm using a dataset of 22,000 records from Kaggle, resulting in an (R^2) of approximately 0.90, MAE of approximately 0.048, and RMSE of approximately 0.061. I built a linear regression model as a baseline, and the performance of the Random Forest model completely outperformed the linear model. The metrics are not linear, so the linear model is unsuitable here. Through the analysis, we can also see that the mental fatigue score (calculated from the CBI test results) is the main parameter influencing (almost 80%) the burnout rate.

However, there are still some limitations in this thesis. Most employee data is simulated (automatically generated) because this information is private and cannot be obtained from real factories or companies. The web-framework still does not have adequate tools for both test takers and administrators. The AI model is only trained with a small number of features and tasks. Even so, this system still does a good job as a web framework with a model capable of predicting early burn out rates with appropriate results.

Keywords: employee burnout detection, Copenhagen Burnout Inventory, Random Forest regression, microservices architecture, industrial workforce, web-based health monitoring.

CHAPTER I

INTRODUCTION

1.1 Background

Workers in industrial workforces have to work with high intensity, constant shifts, KPI pressure as well as an mentally uncomfortable work environment that can cause them to burn out or become mentally exhausted. The World Health Organization (WHO) has recognized burnout as an occupational phenomenon in the world eleventh revision of the International Classification of Diseases (ICD-11), define it as a syndrome resulting from chronic workplace stress that has not been successfully managed [1].

The global consequence of this problem is hundreds of billions of dollars in annual losses due to lost productivity and health care costs [6]. Looking further, the consequences can go beyond material losses but also human damage. Workers can become less alert, more likely to make mistakes and encounter safety incidents. However, companies still only face this problem passively and only begin to respond when employees show obvious psychological symptoms.

By combining the CBI test and machine learning model, we can have a tool to predict employee burnout rates early, along with a convenient web framework, we can deal with this problem more proactively, react earlier and reduce the damage significantly.

1.2 Motivating Scenario and Use Cases

Instead of a reactive approach like traditional methods, such as HR monitoring and paying attention to employee productivity, mood, attitude, and overall work morale,

this thesis presents a proactive web framework and provides tools for two main use cases: employees self-assessing their mental health, and administrators monitoring the overall mental health status of all company employees or departments in real-time, along with tools to respond to high-risk situations.

1.3 Problem Statement

Although burnout severely impacts worker health and industrial productivity, current workplace practices still suffer from several major shortcomings. Most companies only react to burnout after damage has occurred through shift cancellations, costly mistakes, or abrupt resignations, completely lacking any early warning systems. Furthermore, while scientifically proven tools like the Copenhagen Burnout Inventory are available with no cost, they are rarely integrated into routine processes, leaving collected survey data to be processed manually and isolated. Traditional methods also fail because they only report existing symptoms instead of predicting future risks, a gap that machine learning can easily overcome by identifying patterns in historical data to estimate stress levels from just a few key inputs. Ultimately, because personal data remains trapped in individual data repositories, HR teams lack the comprehensive tools such as departmental heatmaps and trend charts necessary to make informed, company-wide decisions. This thesis aims to address precisely these issues by building a single web platform that integrates mental health assessments, intelligent predictive technology, and clear data visualization to help organizations protect their teams before burnout occurs.

1.4 Scope and Objectives

The scope of this thesis focuses on the design, building, and testing of a desktop web framework for early detection of burnout in industrial workplaces. Because the

actual factory records are private and confidential, the system relies on simulated personal data (auto-generated), using the Copenhagen Burnout Inventory as the core survey tool and using a Random Forest regression model trained on a publicly available dataset to predict burnout rate. To achieve this, the project focuses on six core objectives. First, it converts employee responses from online surveys into clear mental fatigue scores that the machine learning system can read. Next, it used that Random Forest model to predict employee burnout rates based on designation, resource allocation, and their mental fatigue scores. From the user side, the project builds a personal portal where employees can check their stress levels via a simple meter, view historical trend charts, see score analysis, and instantly access mental health support. From the management side, it creates a complete HR command center with color-coded departmental heatmaps, risk distribution charts, key performance indicators, and a secure way to closely examine individual employee trends without compromising privacy. This dashboard is supported by an automated alert system that will notify HR immediately when employees reach dangerous stress levels or experience spikes between tests. Finally, the entire platform is connected by a neat microservice architecture, seamlessly separating the Python machine learning model and FastAPI from the Node.js and Express backend, as well as the React and Next.js user interfaces.

1.5 Research Questions

This thesis investigates the following research questions:

- RQ1.** How effectively can a Random Forest regression model predict employee burn rate using only three input features (designation, resource allocation, and mental fatigue score)?
- RQ2.** Can a web-based system provide actionable burnout insights to both employees

(through self-monitoring tools) and HR administrators (through department-level analytics and automated alerts)?

RQ3. What are the practical benefits and trade-offs of using a microservice architecture to separate machine learning inference from the application backend in a health monitoring system?

1.6 Assumptions and Proposed Solution

This system is built with a three-tier web architecture and the following assumptions. First, because employee information is confidential and private to businesses, the employee information in this thesis is simulated and automatically generated. These attributes may be based on attributes in the dataset (designation, gender, date of birth, etc.) or simply simulate real-world scenarios (KPIs, name, etc.). Currently, the application is designed only for desktop users and the language is limited to English. Furthermore, the machine learning model is not overly complex and is trained on a relatively small dataset with limited diversity in terms of fields, countries, geographical locations, and cultures.

To meet these requirements, the technical solution divides responsibilities across three separate layers. The UI/UX layer is a React and Next.js application providing separate, customized portals for employees and administrators, communicating with the backend via RESTful API calls while using TanStack Query to optimize API calls and data caching. The core business logic is handled by the Express backend, which takes care of user authentication, test scoring, burnout risk classification, triggering automatic alerts, and aggregating metrics on the HR dashboard using a combination of MySQL and MongoDB. Finally, machine learning capabilities are completely isolated within a Python microservice and FastAPI, loading a pre-trained Random Forest model before startup to handle real-time stress prediction. This calculated division of responsibilities

allows the backend to switch between a live AI model and a simple code formula using environment variables, giving developers the flexibility to build, test, and deploy each part of the platform completely independently.

1.7 Thesis Structure

This thesis is organised into six chapters, as illustrated in Figure 1.

Chapter I: Introduction presents the background, motivating scenario, problem statement, objectives, research questions, and assumptions.

Chapter II: Literature Review surveys the academic foundations of burnout in industrial workplaces, the Copenhagen Burnout Inventory, machine learning methods for burnout prediction, web application architectures, and related work.

Chapter III: Methodology and System Design describes the overall system architecture, database design, backend design, AI microservice methodology, and frontend design decisions.

Chapter IV: Implementation and Results presents the design and implementation of each component, the machine learning training results, and documents the frontend with annotated screenshots.

Chapter V: Discussion and Evaluation analyses model performance, reports functional testing outcomes, evaluates system features against the stated objectives, answers the research questions, and compares the system with related work.

Chapter VI: Conclusion and Future Work summarises the contributions, assesses the achievement of objectives, discusses limitations, and proposes directions for future development.

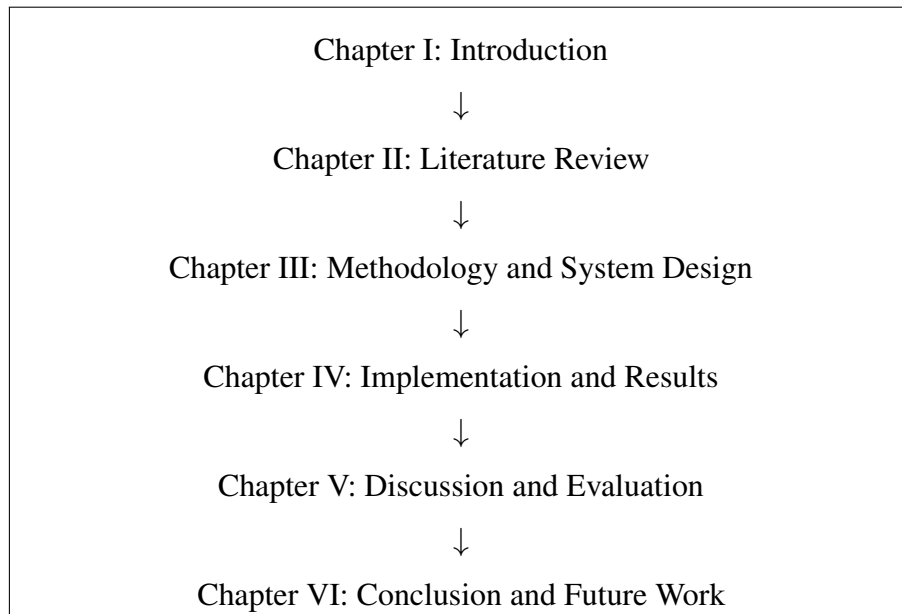


Figure 1: Structure of the thesis.

CHAPTER II

LITERATURE REVIEW

2.1 Occupational Burnout

2.1.1 Definition and Impact on the Industrial Workforce

The concept of occupational burnout was first introduced by Freudenberger in 1974 to describe the state of physical and emotional depletion observed among human service workers [4]. The construct was later formalised by Maslach and Jackson, who defined burnout as a three-dimensional syndrome comprising emotional exhaustion, de-personalisation (cynicism), and reduced personal accomplishment [3]. In 2019 the World Health Organization included burnout in the ICD-11 as an occupational phenomenon, describing it as a syndrome resulting from chronic workplace stress that has not been successfully managed [1].

In industrial and manufacturing environments, the consequences of burnout extend beyond individual suffering. Research has linked burnout to increased absenteeism, higher employee turnover, reduced product quality, and elevated workplace accident rates [7]. A Gallup meta-analysis across industries found that employees experiencing burnout are 63% more likely to take a sick day, 2.6 times more likely to seek a new job, and 13% less confident in their performance [6]. For factory settings with tight production schedules and safety-critical processes, these figures translate directly into operational and financial risk.

2.1.2 Burnout Measurement Instruments

Among the tools for measuring occupational burnout, the most well-known are perhaps the Maslach Burnout Inventory (MBI) test and the Copenhagen Burnout Inventory (CBI) test. The MBI, developed in 1981 and used in numerous studies related to mental health in the workplace, comprises 22 items across three different scales (emotional exhaustion, depersonalization, and personal accomplishment) [3]. Its effectiveness has been proven over time and remains highly effective. The CBI, developed in 2005 by Kristensen et al., focuses on three dimensions: personal burnout, work-related burnout, and client-related burnout [5]. Each aspect is assessed through a questionnaire scored on a five-point frequency scale, from "Always" (100 points) to "Never/Almost Never" (0 points). The CBI has demonstrated high internal consistency (Cronbach alpha coefficient exceeding 0.85 across all subscales) and strong predictive power for outcomes such as sick leave and sleep disorders. Overall, MBI is a widely used and time-tested assessment tool; however, its limitation is that it is copyrighted and requires a commercial license, which is a challenge for low-budget or open-source projects. CBI assessment, on the other hand, is highly effective, free, and readily available to researchers and students needing a reliable tool to assess the mental health of workers in general [5]. Therefore, CBI assessment was chosen for this thesis.

2.1.3 The Copenhagen Burnout Inventory in This Thesis

This thesis uses the CBI for some reasons. First, the CBI is completely free to use, with no licensing restrictions, making it ideal for an academic project. Second, the CBI distinguishes between personal and work-related burnout, which allows the system to provide more granular feedback to both employees and administrators. Third, the

CBI’s straightforward frequency-based response scale maps naturally to a numeric score that can be fed into a machine learning model.

Since the target population consists of industrial workers rather than client-facing professionals, the client-related burnout subscale is omitted. The system uses the remaining 13 questions: six for personal burnout and seven for work-related burnout. One question (“Do you have enough energy for family and friends during leisure time?”) is reverse-scored, as a positive response indicates lower burnout. The section averages are combined and normalised to a 0–10 scale to produce the mental fatigue score used as input to the machine learning model.

2.2 Machine Learning for Health Prediction

2.2.1 Supervised Regression

Machine learning tasks are broadly categorised into supervised and unsupervised learning. In supervised learning, the algorithm is trained on a labelled dataset where each input is associated with a known output [10]. When the output is a continuous numeric value, the task is called regression.

Burnout prediction fits naturally into the supervised regression paradigm: the training dataset contains employee observations with a known burn rate (the target variable), and the goal is to learn a function that maps employee features to a predicted burn rate for new, unseen employees.

2.2.2 Linear Regression

Linear Regression is the simplest regression algorithm. It models the relationship between the input features and the target as a linear equation:

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + b \quad (1)$$

where x_1, x_2, x_3 are the input features (designation, resource allocation, mental fatigue score), w_1, w_2, w_3 are learned weights, and b is a bias term. The weights are optimised by minimising the sum of squared errors between predicted and actual target values [11].

For this thesis, selecting a model that is simple and demonstrates a deep relationship between mental fatigue score and burnout rate is essential to express the relationship between the data and the desired outcome. Therefore, I chose the Linear regression model as the baseline model, and from this, we can also see more clearly the superiority of the Random Forrest regression model compared to the Linear regression model.

2.2.3 Random Forest Regression

Leo Breiman originally introduced Random Forest as an ensemble learning method that constructs a vast collection of decision trees, training each individual tree on a random sample of data and utilizing a random subset of features for every split [12]. The final prediction drops down to a simple average of what all these separate trees conclude. Looking at this specific burnout detection problem, Random Forest brings several major advantages to the table compared to traditional Linear Regression. Because individual trees split data based on local thresholds rather than forcing a single global line across the dataset, the model is highly robust against outliers. It naturally captures complex, non-linear relationships and intricate feature interactions without making you manually tweak the data.

Different feature scales do not affect the model at all. This means variables operating on entirely different ranges, like a designation score from one to five versus a resource allocation score from one to ten, do not require any normalization beforehand.

On top of that, it dishes out built-in feature importance scores, showing exactly how much each specific variable contributes to the final accuracy. For standard tabular datasets that have just a handful of features and tens of thousands of rows, Random Forest consistently delivers strong performance right out of the box with very minimal hyperparameter tuning, making it a highly practical and reliable choice for a thesis project [13].

2.2.4 Model Evaluation Metrics

To evaluate the actual performance of regression models, we need to look at three metrics which are Mean Absolute Error (MAE), Root Mean Squared Error (RMSE) and Coefficient of Determination (R^2). First of all, the MAE give us the average number of the absolute differences between predicted results and actual results, for example, if our burnout rate is a number in range from 0.00 to 1.00 (0% to 1%), the MAE of 0.08 indicate that the model is typically off by 8 percentage. The RMSE is the square root of the mean of squared errors. If the error is big, the RMSE will be very high, making it be specially sensitive to occasional outlier predictions. The final metric is Coefficient of Determination (R^2), it is the proportion of variance in the target variable which is explained by the model. For instance, an R^2 of 0.99 indicates that the model explain 99 percent of the variation in burn rates and the values that are closer to 1.0 (100%) indicate better fit.

With all three metrics together, we can quickly evaluate how good the model is. To summarize, MAE is for average-case accuracy (lower is better), the RMSE for worst case sensitivity (lower is better) and the R^2 for overall accuracy performance of the model [11] (higher is better).

2.3 Web Application Architectures

2.3.1 Microservice vs Monolithic Architecture and why I choose Microservices

Monolithic architecture allow developers build everything (user interface layer, business logic layer, data layer, etc.) in just a single deployable unit so that it is simpler to develop at the beginning stage but it would become a remarkable issue when the app demands maintainance and scalability, especially when different concerns have different technology requirements [23].

A microservice architecture decomposes the application into small, independently deployable services (or components). Each service is responsible for a single concern [23]. Services comunicate via protocols like HTTP/REST or gRPC. Because of the isolation attribute of microservices, it is easier for maintaining complex systems with multiple domains and scaling when the system demands more resource.

In this thesis, I chose microservices architecture because the early burnout detection needs a machine learning model which is better to be developed and built in Python. The AI service should be separated from the ExpressJS backend which handles calculations and accesses to the database. Frontend and backend also use different frameworks, they need to be built, developed and deployed independently.

2.3.2 RESTful API Design

When it comes to wireframing the architecture for networked applications, Representational State Transfer, or REST, serves as the core blueprint [24]. Basically, RESTful APIs just lean on standard HTTP methods like GET, POST, PUT, PATCH, and DELETE to handle whatever operations you need on resources, which are all mapped out by specific URLs. A few big rules keep this whole setup running smoothly. Statelessness is

a major one, meaning every single request has to carry all the data needed to process it on its own, without relying on the server remembering past interactions. You also have the uniform interface constraint and a strict client-server separation to keep things decoupled.

For this thesis, I chose to stick to these RESTful conventions for both the frontend-to-backend communication and the backend-to-AI-service links. To stop client-side error handling from becoming a complete mess, every single response wraps its own data in a consistent JSON envelope format so that it always boils down to either a `success`, data structure or a `success`, `error` fallback, making things way more predictable and easier to parse.

2.4 Related Technologies

2.4.1 React and Next.js for Frontend Development

React stands as a major JavaScript library for assembling user interfaces, and it does this by breaking everything down into a component-based architecture [16]. You basically build with these reusable, self-contained pieces that grab data through props and handle their own local state using hooks. Instead of reloading everything, React utilizes a virtual DOM to surgically update only the exact pieces of the page that actually changed. This is honestly a lifesaver for performance, especially when you are trying to render data-heavy dashboards without lagging the browser.

Then you have Next.js, which is a full-stack framework built on top of React that introduces things like file-based routing, server-side rendering, static generation, and built-in API routes right out of the box [17]. For the implementation in this thesis, I went with Next.js using the App Router, so the file system handles automatic route generation entirely on its own, and I plugged in Turbopack to keep development builds fast. On top

of that, TanStack Query takes care of the server state. It handles the heavy lifting for automatic caching, background refetching, and makes sure the cache invalidates properly whenever a mutation happens.

2.4.2 Node.js and Express for Backend Services

Node.js is a JavaScript runtime built on Chrome's V8 engine that enables server-side JavaScript execution [18]. Its event-driven, non-blocking I/O model makes it well-suited for handling concurrent API requests.

Express is a minimal web application framework for Node.js that provides routing, middleware composition, and HTTP utility methods [19]. This thesis uses Express 5 with a layered architecture (routes, controllers, services, repositories) to maintain separation of concerns. Middleware handles cross-cutting concerns including authentication (JWT verification), request validation (Zod schemas), and error handling.

2.4.3 FastAPI for Machine Learning Serving

FastAPI is a modern Python web framework that is built specifically for spinning up APIs that come with automatic, interactive documentation right out of the box [20]. It relies heavily on Python type hints and Pydantic models to handle request validation automatically, plus it supports asynchronous request handling via ASGI. When you combine its high performance with automatic input validation and auto-generated Swagger docs, it becomes an incredibly good choice for serving machine learning models as standard HTTP endpoints. For the actual setup in this thesis, the FastAPI service loads a scikit-learn model directly at startup, allowing it to spit out predictions with response times consistently dropping below 10ms.

2.5 Related Work

A lot of different systems and research papers have tackled employee burnout detection and workplace wellness monitoring. Most of them branch off from what this thesis actually tries to do in terms of scope or execution.

For example, Pradhan and Jena (2019) built a machine learning framework to predict burnout using survey data, and tested algorithms including logistic regression and support vector machines [9]. The catch is that their work was aimed at binary classification only, only labeling people as burnt out or not, not continuous prediction. Plus, they didn't actually deploy it as a web application. Then you have Bianchi et al. (2021), who did a systematic review of burnout measurement tools and their psychometric properties, which basically just confirmed that the CBI is a valid research tool [8] and obviously, their paper is purely psychometric and does not touch technology integration at all. On the commercial side, employee wellness platforms like Officevibe, Culture Amp, and Lattice offer survey-based engagement tracking paired with dashboard analytics [29]. But these are generic corporate products that usually lack any ML-based prediction capabilities, and they are built for general engagement rather than tracking clinical or industrial burnout. Even the Kaggle community notebooks using the "Are Your Employees Burning Out?" dataset have played around with different regression models for burn rate prediction [15], they show how to train a model in isolation, sure, but they never integrate those predictions into a working web app or provide any sort of high-level organizational analytics.

This thesis fills that exact gap. It hooks up a validated psychometric instrument (the CBI), a trained machine learning model (Random Forest), and a full-stack web application that serves up both individual and organizational views inside one single

integrated platform. Table 1 breaks down how these approaches stack up against each other.

Table 1: Comparison with related work.

System / Study	ML	Web App	Org. Analytics	CBI
Pradhan & Jena (2019)	Yes	No	No	No
Bianchi et al. (2021)	No	No	No	Yes
Commercial platforms	No	Yes	Yes	No
Kaggle notebooks	Yes	No	No	No
This thesis	Yes	Yes	Yes	Yes

CHAPTER III

METHODOLOGY AND SYSTEM DESIGN

3.1 System Architecture Overview

This whole setup runs on a three-tier microservice architecture that hooks up a React/Next.js frontend, a Node.js/Express backend, and a Python/FastAPI AI microservice, with MySQL and MongoDB databases handling the data storage. Everything talks to each other through RESTful HTTP APIs, wrapping all responses in a standard JSON envelope format to keep communication predictable.

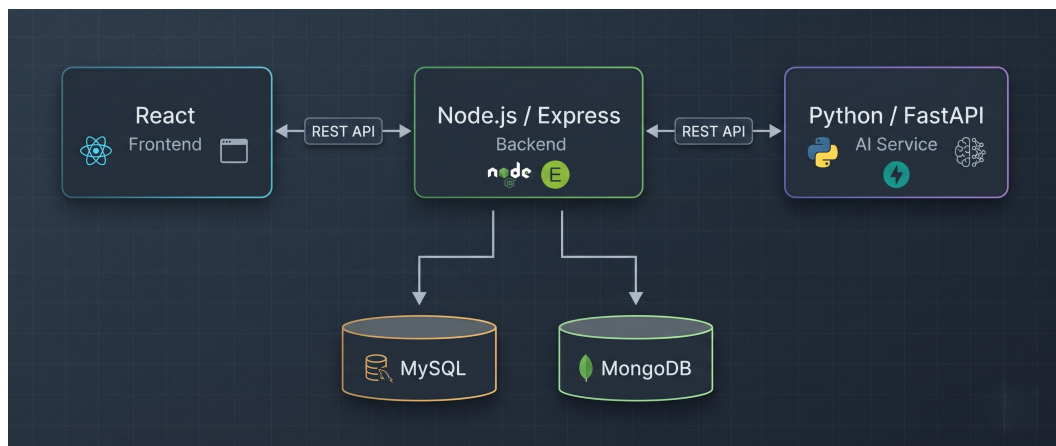


Figure 2: High-level system architecture.

Splitting things into three distinct services boils down to two main reasons. For starters, the machine learning piece absolutely needs Python and scikit-learn, but the web backend handles things better by leveraging JavaScript's massive ecosystem for HTTP routing, database drivers, and middleware. The second big advantage is that the AI service can be developed, tested, and restarted on its own without breaking the main application. To make independent development even easier, I set up the backend to

support two different modes using an environment variable. Setting `AI_USE MOCK=true` triggers a quick in-process mock formula so you don't need the AI service running during basic development, while switching it to `AI_USE MOCK=false` actually routes predictions over to the real Python service. This makes working on separate parts of the stack a lot less painful.

3.2 Database Design

3.2.1 MySQL Schema

For handling all structured relational data, the system relies on MySQL. The underlying schema ties seven tables together using strict foreign key relationships alongside cascading deletes to keep everything clean.

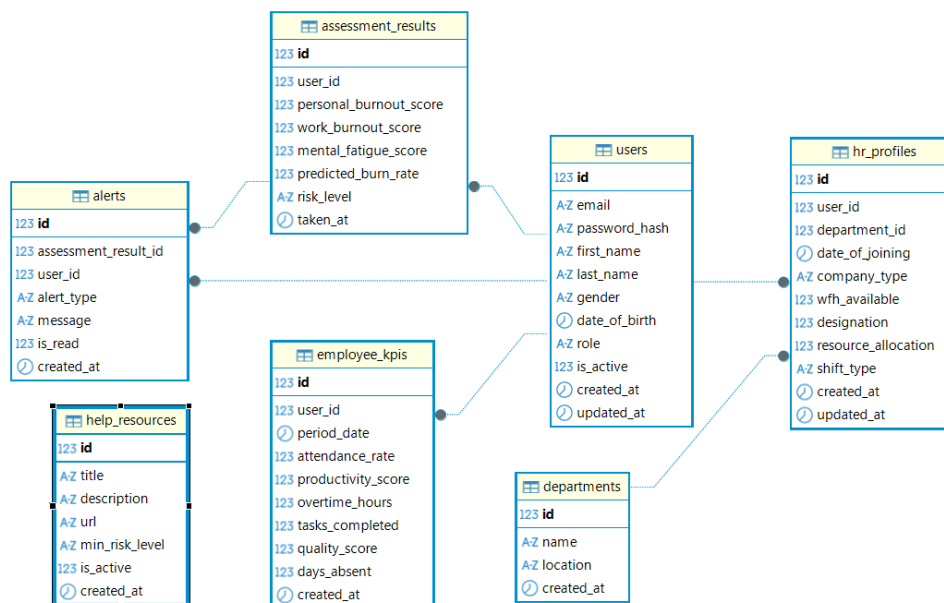


Figure 3: Entity-relationship diagram of the MySQL database.

First up is the **users:** table, which handles authentication by storing bcrypt-

hashed passwords alongside basic demographics and a user's specific role, whether they are an employee or an admin. Then there is **departments**. This functions as a normalized lookup table for the various factory units like Assembly Line A, Maintenance, or Quality Control, and it directly powers the admin heatmap grouping. Tied directly to the user records in a one-to-one relationship is **hr_profiles**. It holds the simulated HR data created right at registration. This includes their department assignment, designation ranging from 1 to 5, resource allocation from 1 to 10, shift type which covers Day, Night, or Rotating, company type, and whether they can work from home. To keep things consistent, these values are pulled straight from the exact same ranges used in the training dataset.

Every time someone finishes a quiz, a new row pops up in the **assessment_results** table. This one tracks a lot of metrics: a personal burnout score from 0 to 100, a work burnout score also from 0 to 100, a mental fatigue score from 0 to 10, a predicted burn rate between 0 and 1, and the final derived risk level. Because the app needs to pull trend queries fast, the table is indexed on (`user_id`, `taken_at`). If an assessment actually crosses a dangerous risk threshold, the system triggers the **alerts** table. It deals with three distinct types, specifically `high_risk`, `critical_risk`, and `trend_spike`. For the unread alert inbox to load without lag, this table uses an index on (`is_read`, `created_at`).

Monthly performance snapshots live in the **employee_kpis** table. It captures a bunch of workplace metrics per employee, including attendance rates, productivity and quality scores, overtime hours, completed tasks, and total days absent. Admins use this on their dashboards to figure out how burnout risk links up with actual job performance. Finally, **help_resources** hosts a curated list of mental health resources. Each resource has a minimum risk level threshold attached to it, meaning the system only displays

active resources if their threshold sits at or below the employee's current risk level.

3.2.2 MongoDB Collections

To handle document-oriented data that really needs schema flexibility, the system brings in MongoDB to store two specific collections.

The first one is **cbi_questions**, which is just a single document holding the whole CBI quiz definition. Inside, it packs the quiz name, version number, five response options with their numeric values, and two section definitions covering personal burnout and work-related burnout. It also includes 13 question objects that track the actual text, section assignment, and a reverse-scored flag. Keeping the entire quiz locked inside one single document is great because it preserves atomicity and completely cuts out the need for slow joins.

Then you have **quiz_submissions**, which drops in one document every single time someone attempts an assessment. This document records the MySQL user ID, the quiz version, and an array of 13 enriched response objects. Each object in that array captures the question ID, answer label, answer value, and the adjusted value if the item happens to be reverse-scored. On top of that, it stores pre-computed section averages and a back-link pointing straight to the corresponding MySQL `assessment_results` row. Going with this structural pattern keeps the raw answers bundled up together as an atomic unit, which is way better than spreading them out across a bunch of relational rows.

3.3 Backend Design

3.3.1 Layered Architecture

For isolating HTTP concerns from actual business logic and data access, the Express backend uses a strict four-layer architecture.

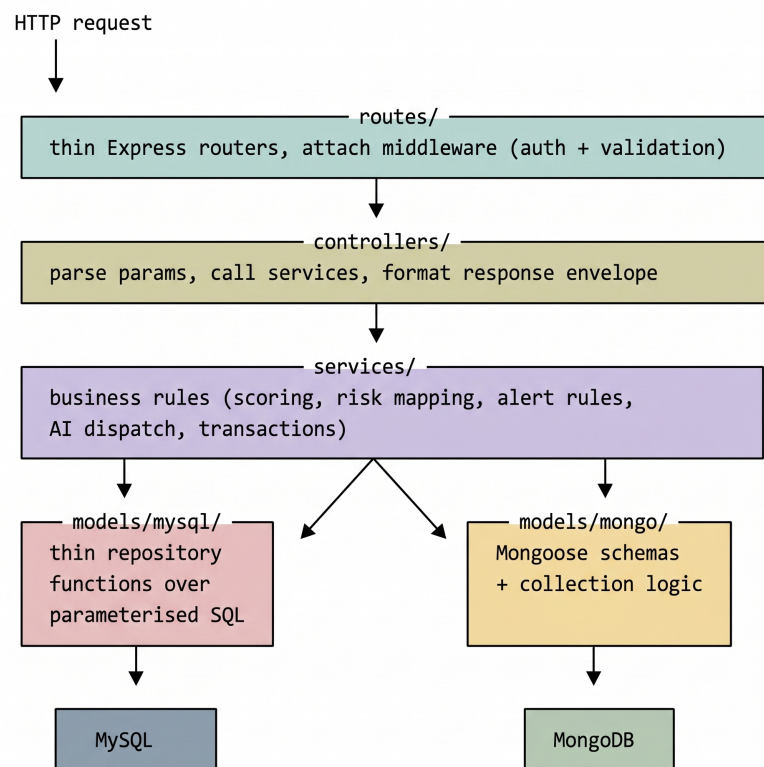


Figure 4: Layered architecture of the Express backend.

Starting at the edge, **Routes** are responsible for defining the HTTP endpoints. This layer hooks up the middleware needed for authentication via `authenticate`, role enforcement through `requireRole`, and request validation using Zod schemas. You won't find any business logic sitting here. Right behind them are the **Controllers**, which are kept pretty thin. Their only job is to grab parameters out of the request object, hand

the heavy lifting over to a service, and then pack the outcome into a uniform success envelope looking like `success: true, data`.

The real brain of the operation lives in the **Services** layer, which handles all domain logic. It runs the CBI scoring including the reverse scoring math, calculates mental fatigue, dispatches calls to the AI service, and maps out risk levels. It is also where alert rule evaluation, dashboard aggregation, and database transactional operations happen. Finally, **Repositories** completely own the database interaction. The MySQL repositories run parameterized SQL queries using `mysql2/promise`, while the MongoDB repositories handle things through Mongoose schemas. Keeping it locked down like this ensures no other layer ever touches the database drivers directly.

3.3.2 Authentication and Authorisation

To keep authentication stateless, the system relies on JSON Web Tokens (JWT). When someone registers, the system hashes their password using `bcrypt` and immediately hands back a signed JWT that packs the user's ID and role inside. To make sure everything is atomic, the HR profile gets built inside the exact same MySQL transaction as the base user record so you don't end up with half-created profiles.

Handling authorization happens right at the router level using role-based access control (RBAC). It is pretty straightforward: employee routes block anything unless `role = "employee"`, admin routes lock things down to `role = "admin"`, and shared routes just let any authenticated user through. Also, to prevent weak configurations, the environment variable schema forces the JWT secret to be at least 16 characters long.

3.3.3 Quiz Scoring Pipeline

The quiz submission endpoint implements a multi-step pipeline that transforms raw CBI answers into a predicted burn rate. The pipeline operates as follows:

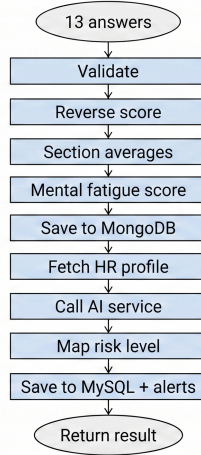


Figure 5: Quiz submission scoring pipeline.

The scoring formulae are:

$$\text{personalBurnoutAvg} = \frac{1}{6} \sum_{i=1}^6 v_i \quad (0-100) \quad (2)$$

$$\text{workBurnoutAvg} = \frac{1}{7} \sum_{i=7}^{13} v_i \quad (0-100) \quad (3)$$

$$\text{mentalFatigueScore} = \frac{\text{personalBurnoutAvg} + \text{workBurnoutAvg}}{20} \quad (0.0-10.0) \quad (4)$$

where v_i is the adjusted answer value for question i (with reverse scoring applied to question 10).

3.3.4 Alert Rules

Right when the assessment result is saved, two specific alert rules get evaluated inside that very same MySQL transaction.

First up is the Threshold alert. If the calculated risk level hits either “high” or “critical”, the system immediately spins up an alert categorized as `high_risk` or `critical_risk`. The second check is the Trend spike alert. This one looks backward: if the current burn rate jumps past the user’s previous assessment burn rate by 0.15 or more, it triggers a `trend_spike` alert type. Because these checks run in parallel, both rules can easily fire off from a single assessment submission, which means you could end up with up to two alert rows written to the database at once.

3.4 AI Microservice Design

3.4.1 Dataset Description

The machine learning model leverages the public Kaggle dataset "Are Your Employees Burning Out?" [15], a dataset capturing roughly 22,000 distinct employee observations. Out of the nine columns available for each row, the training setup utilizes only three features alongside a single target variable.

Table 2: Dataset columns used for model training.

Column	Type	Range	Role
Designation	Integer	1–5	Feature
Resource Allocation	Integer	1–10	Feature (has nulls)
Mental Fatigue Score	Float	0.0–10.0	Feature (has nulls)
Burn Rate	Float	0.0–1.0	Target variable

Interestingly, several attributes like Employee ID, Date of Joining, Gender,

Company Type, and WFH Setup Available are completely bypassed during training, even though they sit right there in the dataset. This tight focus on just three features is actually a deliberate design choice. We had to limit the feature set because the live application only has access to these specific values at prediction time. Specifically, the simulated HR profile supplies the designation and resource allocation metrics, while the mental fatigue score comes straight from the user's CBI quiz results.

3.4.2 Feature Selection Justification

We restricted the model to just three features for three specific reasons. First of all, it keeps the model perfectly in sync with the actual data our application gathers, meaning we do not have to rely on complex features that would require deep, real-world factory integration. On top of that, keeping the feature set small makes the whole model a lot easier to interpret. That is pretty vital for making the thesis defense convincing and building actual trust with the end users. Finally, because the simulated HR values are explicitly generated using the exact same ranges found in the training data, the model always handles in-distribution inputs. This drastically cuts down the risk of getting wild, unreliable predictions.

3.4.3 Data Preprocessing

Our preprocessing pipeline basically boils down to three quick steps. First, we drop the null targets. If a row has a missing Burn Rate, we just chuck it out since there is no ground-truth label to train on. Next up is median imputation. For any missing values in Resource Allocation or Mental Fatigue Score, we swap them out with the column's median. We chose the median over the mean here because it handles outliers much better on these bounded ordinal scales. Last, we handle the train/test split. The dataset gets

split 80%/20%, and we locked it in with a fixed random seed (`random_state=42`) so the results stay completely reproducible.

Looking at how this works during actual inference, the Node.js backend does a bit of safety filtering. It automatically coerces any null `resource_allocation` inputs to a default value of 5, which is just the midpoint of the scale. It does this right before hitting the Python service, guaranteeing that the model always receives all three inputs without any missing data gaps.

3.4.4 Model Selection

We trained and compared two different models for this task. First, a standard Linear Regression serves as our baseline, which basically gives us a performance floor and lets us look directly at the fully transparent learned weights. Then we have our primary model, a Random Forest Regressor. We picked this one because it is great at capturing non-linear interactions, handles outliers without breaking a sweat, features scale invariance, and gives us built-in feature importance out of the box.

We did think about using other alternative algorithms like Gradient Boosting, Neural Networks, SVR, and kNN, but we ended up skipping them. Gradient Boosting methods like XGBoost or LightGBM really only offer tiny, marginal gains over a Random Forest when you are dealing with a tiny 3-feature problem, and they just add a bunch of annoying hyperparameter complexity. Neural networks are overkill here too, requiring way more data and tedious tuning to even match tree-based methods on small tabular datasets. As for SVR, it is way too sensitive to feature scaling, and kNN falls apart with noisy data while offering zero insight into feature importance.

3.4.5 FastAPI Service Architecture

The Python AI microservice functions as a thin HTTP wrapper built right around the serialized model:

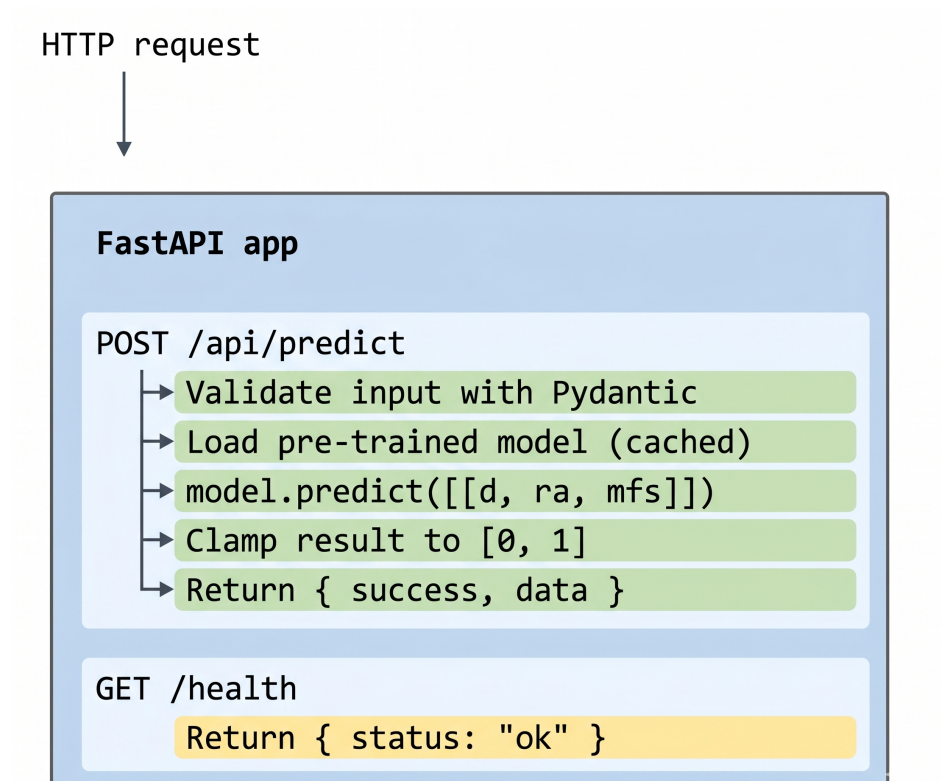


Figure 6: Architecture of the Python/FastAPI AI microservice.

To keep things fast, the application loads the actual model file (`burnout_model.pkl`) exactly once during startup using a FastAPI lifespan handler, caching it immediately in a module-level variable. Because every single incoming prediction request reuses this exact in-memory model, we completely bypass slow disk I/O operations and successfully keep our response times under 10 milliseconds. On top of that, Pydantic handles strict input validation right at the API boundary. If an incoming request contains any out-of-range or missing values, the system catches it immediately and throws a 422

error before it can even touch the model.

3.5 Frontend Design

3.5.1 Technology Stack and Design Decisions

For the frontend, we decided to go with Next.js 16 using the App Router and React 19, keeping everything strictly typed with TypeScript, and we rely on TanStack Query to manage caching, background refetching, and cache invalidation whenever a mutation happens. All of our forms are hooked up to react-hook-form and validated using Zod schemas. For the data visuals, we chose Recharts to render the charts, and we grab our icons from lucide-react.

One big architectural shortcut we took is that the application is built strictly for desktop screens. Since the primary target environments are HR office terminals and factory kiosk stations, we completely skipped implementing mobile breakpoints to save time.

3.5.2 Route Structure and Navigation

The entire frontend architecture maps out to 13 specific routes, which we split cleanly into three buckets: public authentication routes, employee portal pages protected by the `role = "employee"` requirement, and admin portal screens locked behind the `role = "admin"` permission.

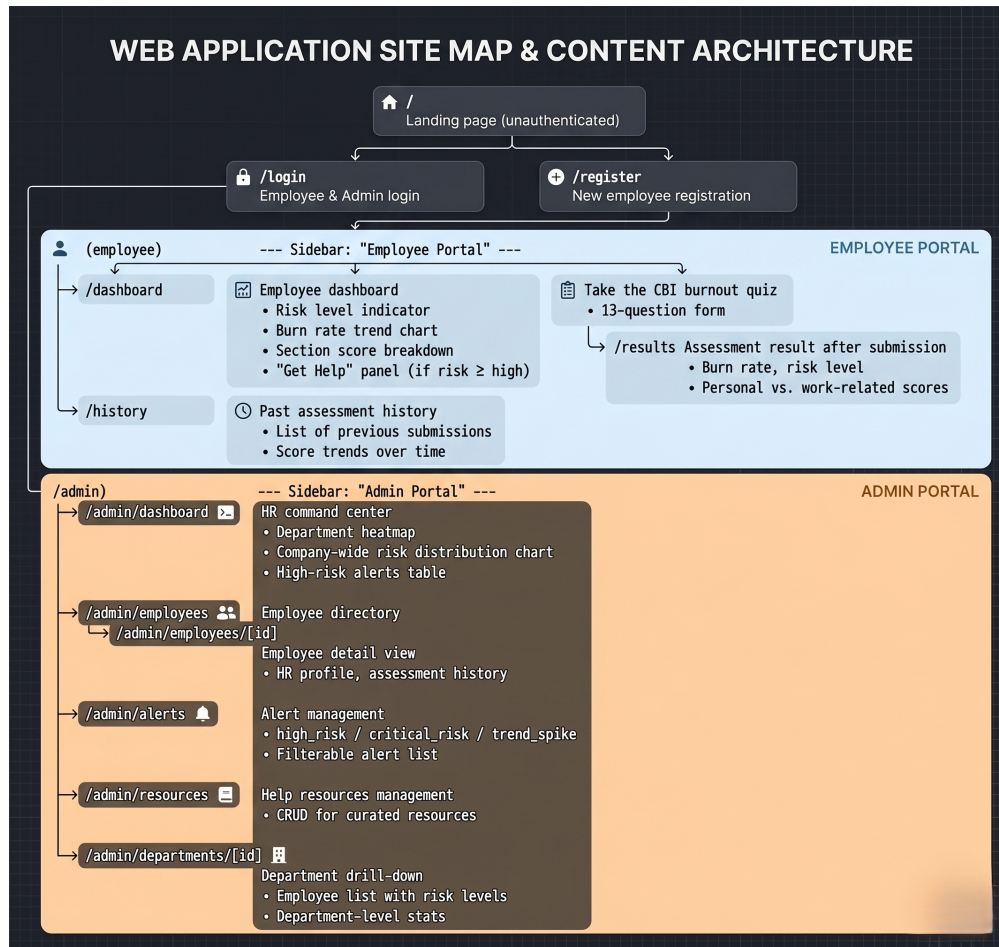


Figure 7: Application route structure and navigation flow.

To enforce this setup, each portal layout wraps its children inside a custom `RouteGuard` component. This component intercepts the navigation to verify the user's current authentication status and specific role. If someone unauthenticated tries to sneak in, it boots them back to `/login`, and if an authenticated user tries to access the wrong portal, it automatically redirects them right back to their own designated dashboard.

3.5.3 Design System

Our visual identity uses a warm, paper-like color palette that we encoded entirely as Tailwind v4 CSS custom properties. We made sure every single color used across the application maps back directly to a named design token, so you cannot find any random, ad-hoc hex values thrown around the component code. You can check out Table 3 to see how the primary tokens are laid out.

Table 3: Primary design tokens.

Token	Hex	Role
parchment	#f5f4ed	Page background
ivory	#faf9f5	Card surfaces
terracotta	#c96442	Primary accent / CTA
near-black	#141413	Primary text
risk-low	#6b8f5f	Low risk indicator
risk-moderate	#c69a3d	Moderate risk indicator
risk-high	#c96442	High risk indicator
risk-critical	#b53333	Critical risk indicator

For the typography, we went with a Georgia serif typeface for the headlines to give the app a more formal, document-like vibe, but kept the body text and UI elements clean using the Inter sans-serif family. We also established four distinct risk-level colors, which makes it super easy to get an instant visual classification whenever they pop up on charts, badges, and status indicators throughout the interface.

3.5.4 Data Flow Pattern

We handled all server communication by sticking to a strict, four-layer architecture pattern to keep things uniform across the codebase.

First, the actual Page component sits at the top, rendering the UI and pulling

in a custom TanStack Query hook. That custom hook (`useXxx`) acts as the second layer, wrapping around `useQuery` or `useMutation` to manage things like cache keys, enabled conditions, and invalidation rules. Below that, the third layer is the API function (`lib/api/`), which triggers a strongly typed `apiFetch()` call pointing to the right endpoint path and arguments. Finally, everything funnels into the base Fetch client (`lib/api/client.ts`). This low-level client automatically tacks on the base URL, injects the user's JWT directly from `localStorage`, unpacks the standard `success`, `data` response envelope, and throws a typed `ApiError` if anything breaks. If it intercepts a 401 Unauthorized status, it instantly wipes the bad token and boots the user back to the login page.

For anything altering data, we just hook up mutations using `useMutation` and trigger cache invalidation the second they succeed. This keeps the UI completely fresh and up to date without forcing annoying full-page reloads.

CHAPTER IV

IMPLEMENTATION AND RESULTS

4.1 Development Environment

Table 4 summarises the tools and technologies used during development.

Table 4: Development tools and versions.

Tool	Version	Purpose
Visual Studio Code	Latest	Code editor and IDE
Node.js	≥ 18.17	Backend runtime
Python	3.12	AI service runtime
MySQL	8.x	Relational database
MongoDB	7.x (Atlas)	Document database
Git	Latest	Version control
npm	≥ 9	Package management (JS)
pip	Latest	Package management (Python)
Jupyter Notebook	Latest	EDA and model training

4.2 Backend Implementation

4.2.1 Project Structure and Configuration

The backend codebase mirrors the exact folder layout we established back in Section 3.3.1, drawing sharp lines between configuration, middleware, models, services, controllers, routes, validators, and utility scripts. To stop configuration bugs before they happen, all environment variables run through a Zod validation schema inside `config/env.js` the moment the application kicks off. If a required variable is missing or formatted wrong, the process immediately crashes with a clear error message, ensuring

the server never boots up in a broken, half-configured state.

When the server spins up, it connects to the databases in a strict chain reaction. First, `server.js` pings MySQL, jumps over to connect to MongoDB next, and only opens up the HTTP listener once both databases are happy. If any link in this chain breaks, the whole startup sequence aborts on the spot. We also hooked up SIGINT and SIGTERM event listeners to manage graceful shutdowns, meaning the app will systematically stop the HTTP server first before safely closing down both database connections.

4.2.2 Authentication Flow Implementation

Whenever a new user registers, the backend handles their account creation and spits out a synthetic HR profile simultaneously inside a single MySQL transaction block.

First, Zod parses and validates the request body to check the email, password, name, gender, and date of birth. Then, bcrypt hashes the password using our configured salt rounds before the system inserts a new row into the `users` table. Right after that, the code cooks up a simulated HR profile by randomly picking a department and generating mock metrics for the user's designation (1-5), resource allocation (1-10), shift type, company type, WFH availability, and a fake joining date. This profile gets saved directly into the `hr_profiles` table. Finally, the app signs a JWT containing the new user's ID and role, sending that token and the user data back to the client.

Wrapping this entire sequence in a database transaction is super important here, because if the HR profile generation throws an error, the database rolls back completely and prevents an orphaned user record from clogging up our tables. Logging in works a lot more simply, just checking incoming credentials against the stored bcrypt hash and handing out a fresh JWT with an expiration that defaults to 7 days.

4.2.3 Quiz Scoring Implementation

The backend relies on the quiz submission handler inside `quiz.service.js` to run the full pipeline we laid out back in Section 3.3.3. First, it goes through basic answer validation. The system expects exactly 13 answers, checks that all the question IDs are actually valid, flags any duplicate IDs, and forces the answer values to fit into the specific set `{0,25,50,75,100}`. Then there is Question 10, which asks, “Do you have enough energy for family and friends during leisure time?”. This one requires reverse scoring, so we use `adjustedValue = 100 - rawValue`, keeping both the original and adjusted values inside the enriched response object. For data safety, we chose a MongoDB-first persistence approach. The raw submission hits MongoDB before we even try making the AI call, so we do not lose user answers if things crash later. If the next steps fail, the system just cleans up that orphan document. Finally, saving the assessment results and checking the alert rules happens inside a single, transactional MySQL write. Once that goes through, the MongoDB document gets back-linked with the new MySQL assessment ID.

4.2.4 Dashboard Aggregation

To load the admin dashboard, a single API call figures out all the company-wide and department-level stats. We used a correlated subquery to pull the most recent `assessment_results.id` for each employee so we only look at their latest data. JavaScript then takes this result set and aggregates it to build out a few things. It calculates the company overview, which covers total employees, total assessments, average burn rate, and how the risk numbers are distributed. It also handles the department breakdown by looking at employee counts, average burn rates, high-risk numbers, and risk

distributions. On top of that, it generates department KPIs by averaging out attendance, productivity, and quality scores, and it grabs the 10 most recent alert records. If an admin uses the department drill-down endpoint, they get server-side pagination, sorting, and filtering by things like risk level, shift type, or designation.

4.2.5 Alert System Implementation

Right after an assessment finishes, the alert service runs the two rules we talked about in Section 3.3.4. The system runs both the threshold check and the trend spike comparison right inside that same MySQL transaction that handles the assessment result. For the trend spike threshold, the default is 0.15, but you can change this whenever you want by tweaking the `ALERT_TREND_SPIKE_DELTA` environment variable.

4.2.6 API Endpoint Summary

We put together Table 5 to give a quick summary of all the RESTful endpoints the backend exposes. If you need the full API reference with all the specific request and response schemas, it can be found in Appendix A.

Table 5: Summary of backend API endpoints.

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Create account
POST	/api/auth/login	Public	Authenticate
GET	/api/auth/me	Any	Current user
GET	/api/quiz	Employee	Fetch CBI quiz
POST	/api/quiz/submit	Employee	Submit answers
GET	/api/quiz/history	Employee	Past submissions
GET	/api/dashboard/employee	Employee	Dashboard data
GET	/api/resources	Employee	Help resources
GET	/api/dashboard/admin	Admin	Command centre
GET	/api/dashboard/admin/dept/:id	Admin	Department detail
GET	/api/.../dept/:id/analytics	Admin	Dept. analytics
GET	/api/.../admin/employees	Admin	Employee list
GET	/api/.../employees/:id	Admin	Employee detail
GET	/api/alerts	Admin	List alerts
PATCH	/api/alerts/:id/read	Admin	Mark alert read
PATCH	/api/alerts/read-all	Admin	Mark all read
GET	/api/admin/resources	Admin	List resources
POST	/api/admin/resources	Admin	Create resource
PUT	/api/admin/resources/:id	Admin	Update resource
DELETE	/api/admin/resources/:id	Admin	Delete resource

4.3 AI Service Implementation

4.3.1 Model Training Results

We split our preprocessed dataset using a standard 80/20 train/test split, leaving us with around 22,000 rows to work with after stripping out any null targets. To see how well our setups actually performed, we tested them on the held-out test set. You can see the full breakdown of the evaluation results in Table 6.

Table 6: Model performance comparison on the test set.

Model	MAE	RMSE	R^2
Linear Regression	0.0539	0.0716	0.8656
Random Forest	0.0480	0.0611	0.9019

Looking at the numbers, the Random Forest model hit an R^2 of 0.90, which basically means it manages to explain 90% of the variance in the burn rate data. It also got an MAE of 0.048. In plain terms, that tells us the predictions usually miss the real values by less than 5 percentage points when you map it on that 0–1 scale. It clearly beat Linear Regression across all three metrics, giving us a solid 3.6 percentage-point jump in R^2 .

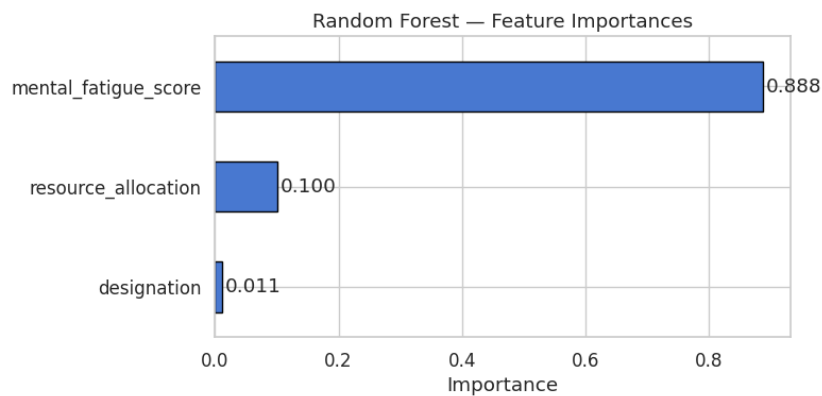


Figure 8: Random Forest feature importance scores.

When looking at the feature importance analysis shown in Figure 8, the mental fatigue score completely dominates the predictions. It carries about 85% of the model’s total predictive power. This makes total sense if you think about how burnout works in the real world. A worker who is completely exhausted psychologically is going to face a high risk of burnout, and things like how senior they are or how heavy their workload is won’t change that much.

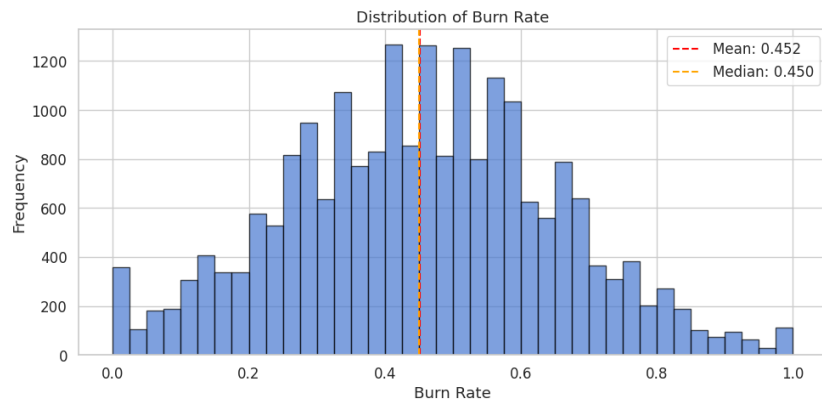


Figure 9: Distribution of Burn Rate values in the training dataset.

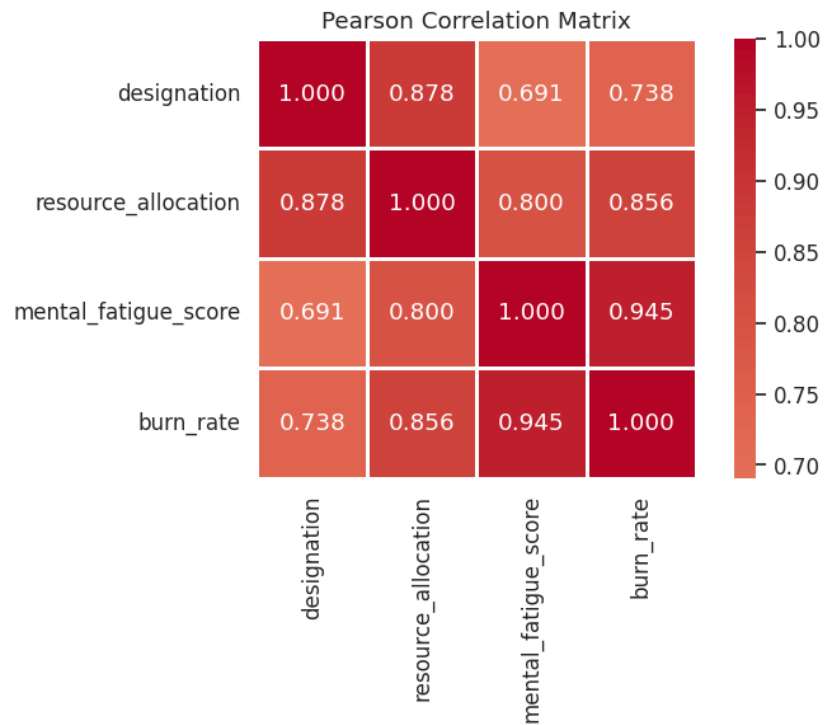


Figure 10: Pearson correlation heatmap of features and Burn Rate.

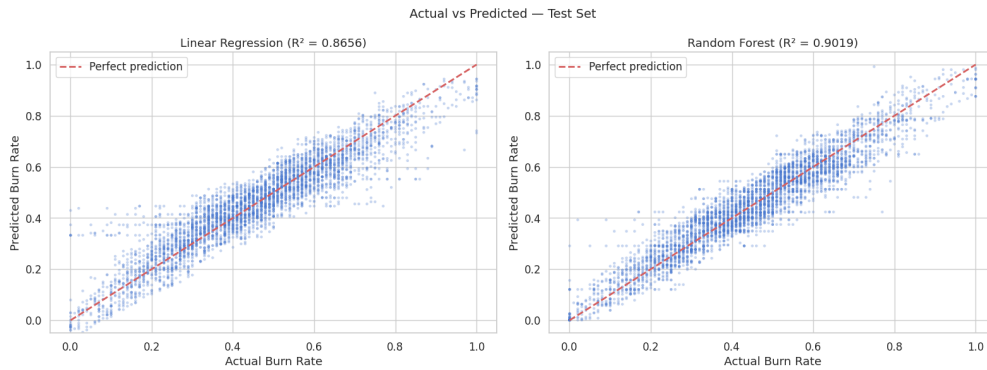


Figure 11: Actual vs. predicted burn rate for both models.

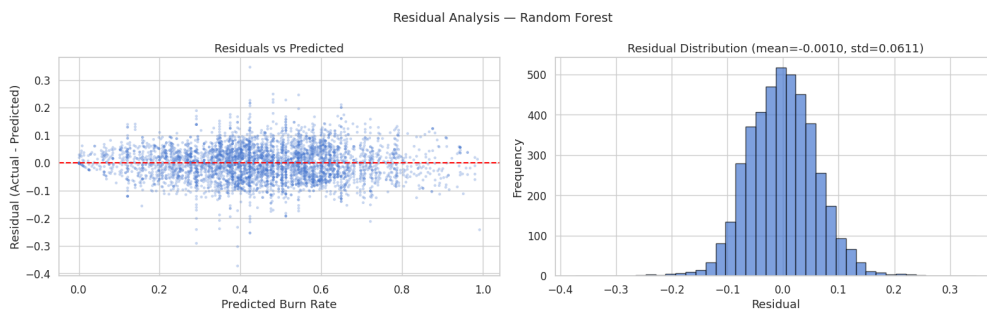


Figure 12: Residual analysis for the Random Forest model.

4.3.2 FastAPI Prediction Endpoint

For the prediction endpoint, we used FastAPI and hooked it up with Pydantic models to handle input validation. It enforces some strict boundaries on the incoming data: designation has to be between 1–5, resource allocation ranges from 1–10, and the mental fatigue score goes from 0–10. To keep things fast, a lifespan handler loads the machine learning model once right at startup and caches it directly in memory. When a prediction request hits the server, the code sets up the feature array, fires off `model.predict()`, clamps the output so it stays within the valid `[0, 1]` range, and sends back the burn rate inside our standard envelope format. The best part is that response times are incredibly quick, consistently clocking in under 10 milliseconds.

4.3.3 Integration with Express Backend

Over on the Express side, `ai.service.js` handles things using a dual-mode setup that depends entirely on the `AI_USE MOCK` environment variable. When you turn this variable to `false`, the service throws an HTTP POST request at the Python microservice with those three features and grabs `response.data.predictedBurnRate`. We also wrapped the request in an `AbortController` using a configurable timeout variable called `AI_SERVICE_TIMEOUT_MS`. If the connection times out or the server returns a non-2xx status, the code intercepts it, formats it as an `ApiError.internal()`, and sends a generic 500 error up to the frontend.

Now, if you flip that variable to `true`, the backend skips the Python call entirely and uses this mock formula locally:

$$\text{burnRate} = \text{clamp}_{[0,1]}(0.07 \cdot \text{mfs} + 0.03 \cdot d + 0.02 \cdot ra - 0.05 + \text{noise}) \quad (5)$$

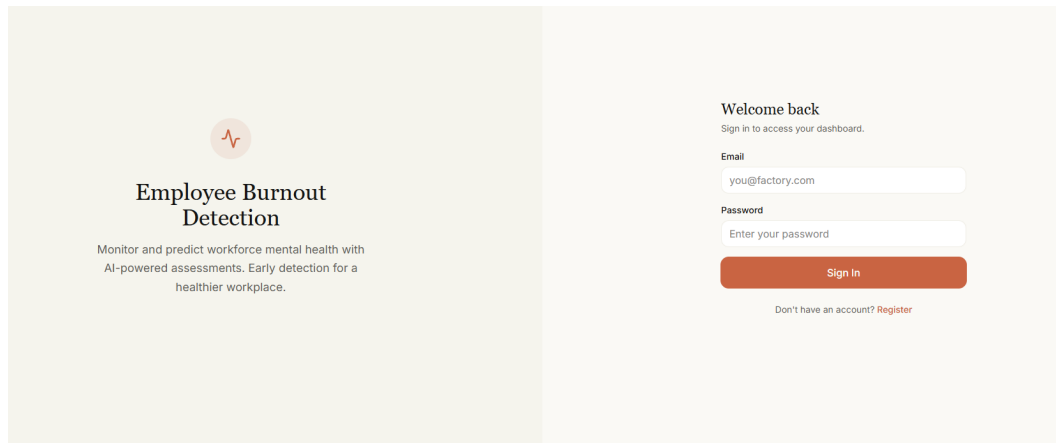
here, *mfs* represents the mental fatigue score, *d* stands for designation, *ra* maps to resource allocation, and *noise* injects a bit of uniform random variance between $[-0.025, +0.025]$. It makes testing a lot easier since jumping between real AI predictions and the mock math requires zero code refactoring, you just tweak the environment variable and bounce the server.

4.4 Frontend Implementation

4.4.1 Authentication Pages

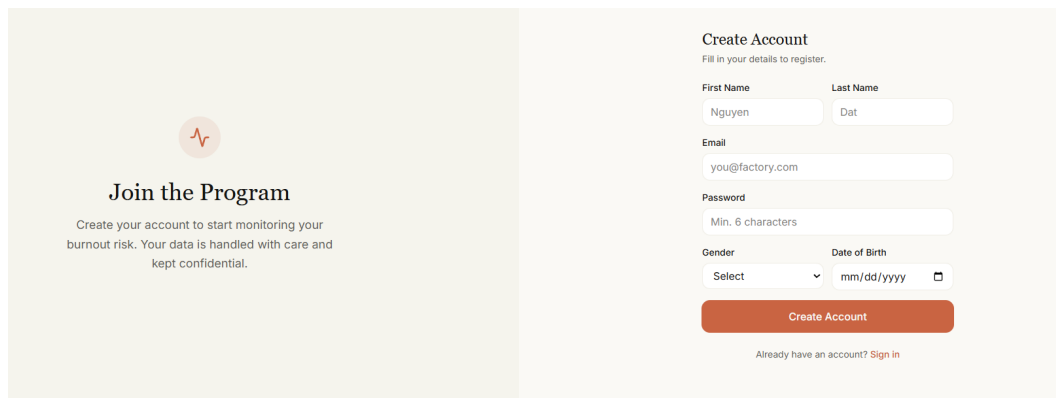
We went with a clean split-screen look for both the login and registration pages. The left side holds a brand illustration paired with a serif headline over a nice parchment

background, while the actual form sits on an ivory surface on the right side. To keep the forms responsive and secure, we managed them with react-hook-form and backed them up with Zod validation schemas, which catches any input errors on the client side before anything even gets submitted.



The login page features a split-screen layout. The left side has a light beige background with a red heart icon and the text "Employee Burnout Detection" followed by "Monitor and predict workforce mental health with AI-powered assessments. Early detection for a healthier workplace." The right side is a white panel with the heading "Welcome back" and "Sign in to access your dashboard." It contains input fields for "Email" (with "you@factory.com" entered) and "Password" (with "Enter your password" as a placeholder). A red "Sign In" button is at the bottom, with a link "Don't have an account? Register" below it.

Figure 13: Login page with split-screen layout.



The registration page also uses a split-screen layout. The left side is identical to the login page. The right side is a white panel with the heading "Create Account" and "Fill in your details to register." It includes input fields for "First Name" (with "Nguyen" entered), "Last Name" (with "Dat" entered), "Email" (with "you@factory.com" entered), and "Password" (with "Min. 6 characters" as a placeholder). There are also dropdowns for "Gender" (with "Select" as the placeholder) and "Date of Birth" (with "mm/dd/yyyy" as the placeholder). A red "Create Account" button is at the bottom, with a link "Already have an account? Sign in" below it.

Figure 14: Registration page.

4.4.2 Employee Portal

The employee portal is built around a sidebar navigation menu that gives users quick access to four primary pages: Dashboard, Assessment, Results, and History.

Employee Dashboard (/dashboard). Think of this as the central hub for the user. It starts by displaying the employee’s basic profile info. Right next to that, we built a custom SVG semicircle gauge with a moving needle and color-coded risk labels to visually represent their current burn rate. There is also a bar chart that compares personal versus work-related burnout scores on a 0–100 scale, alongside a line chart tracking how their burn rate has moved over time. We overlaid this line chart with reference lines at the 0.35, 0.55, and 0.80 risk thresholds to make reading the trends obvious. If someone hits a high or critical risk level, the dashboard dynamically renders a special “Get Help” panel packed with mental health resources. On the flip side, if a new user logs in and hasn’t filled out any assessments yet, the app displays a clean empty state with a big call-to-action button prompting them to take their very first quiz.

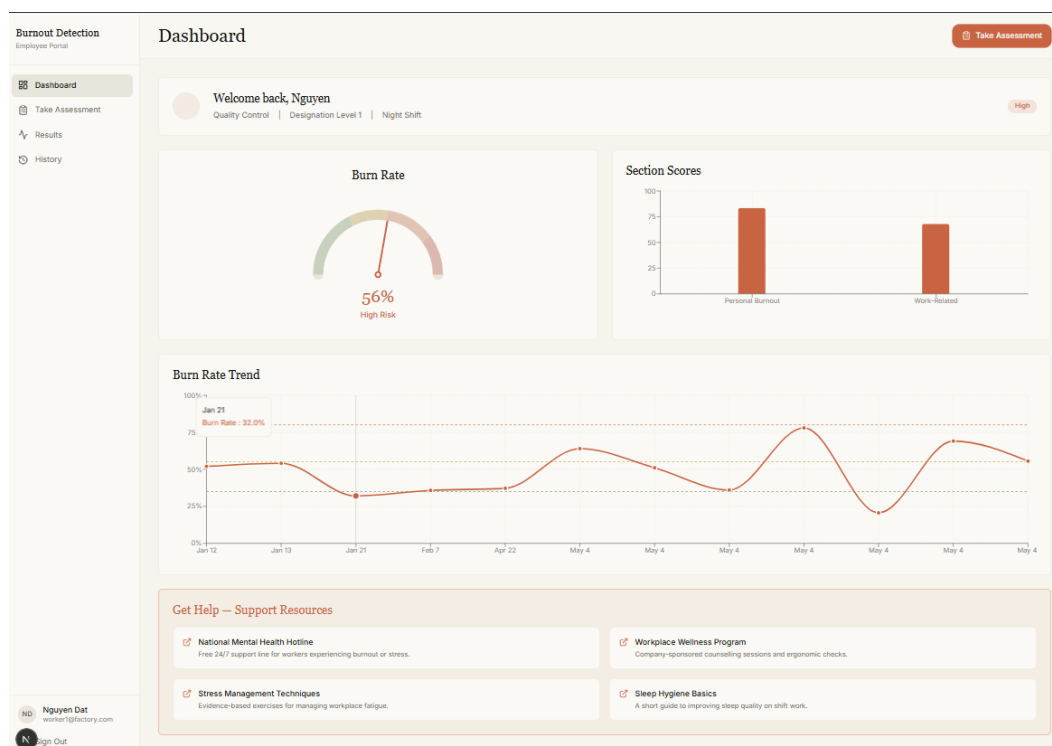


Figure 15: Employee dashboard with burn rate gauge and trend chart.

CBI Assessment (/assessment). A two-step wizard presents the 13 CBI questions. Step 1 covers Personal Burnout (questions 1-6) and Step 2 covers Work-Related Burnout (questions 7-13). Each question displays five large clickable radio card options (Always, Often, Sometimes, Seldom, Never/almost never). A progress bar shows the current step. The “Next” button is disabled until all questions in the current step are answered. On submission, the backend scores the quiz, calls the AI service, and redirects the user to the results page.

The screenshot shows the 'Burnout Detection Employee Portal' interface. On the left is a sidebar with navigation links: 'Dashboard', 'Take Assessment' (highlighted), 'Results', and 'History'. The main content area is titled 'Assessment' and shows 'Step 1 of 2' with a progress indicator '0 / 13 answered'. The section is titled 'Personal burnout'. It contains three numbered questions, each with five radio button options: 'Always', 'Often', 'Sometimes', 'Seldom', and 'Never/almost never'. The questions are: 1. 'How often do you feel tired?', 2. 'How often are you physically exhausted?', and 3. 'How often are you emotionally exhausted?'. The interface is clean and modern, using a light beige color scheme.

Figure 16: CBI assessment page — Step 1: Personal Burnout.

Results (/results). Displays the outcome of the most recent assessment: a large burn rate gauge with risk-level badge, three score cards (personal burnout, work-related burnout, mental fatigue), and support resource cards filtered by the employee’s current risk level. Action buttons allow retaking the assessment or viewing history.

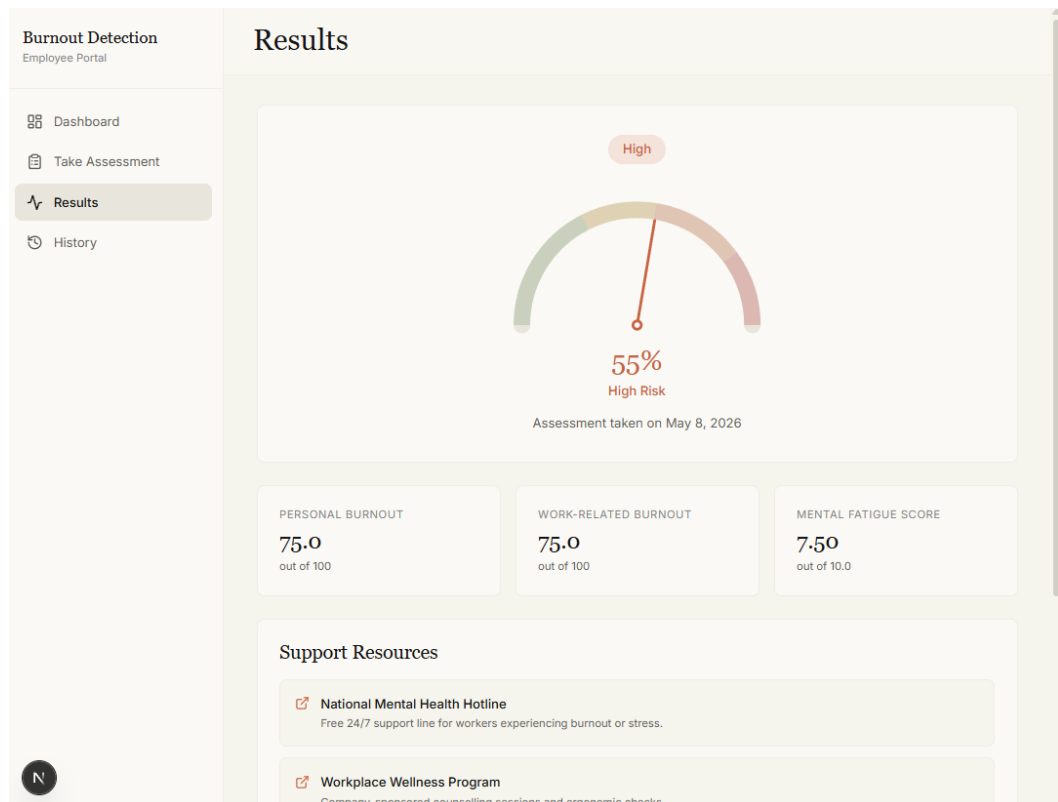


Figure 17: Assessment results page with score cards and support resources.

History (/history). A paginated table listing all past submissions with date, personal burnout average, work burnout average, and quiz version. Each row is expandable to reveal the 13 individual responses with labels and values; reverse-scored answers are annotated.

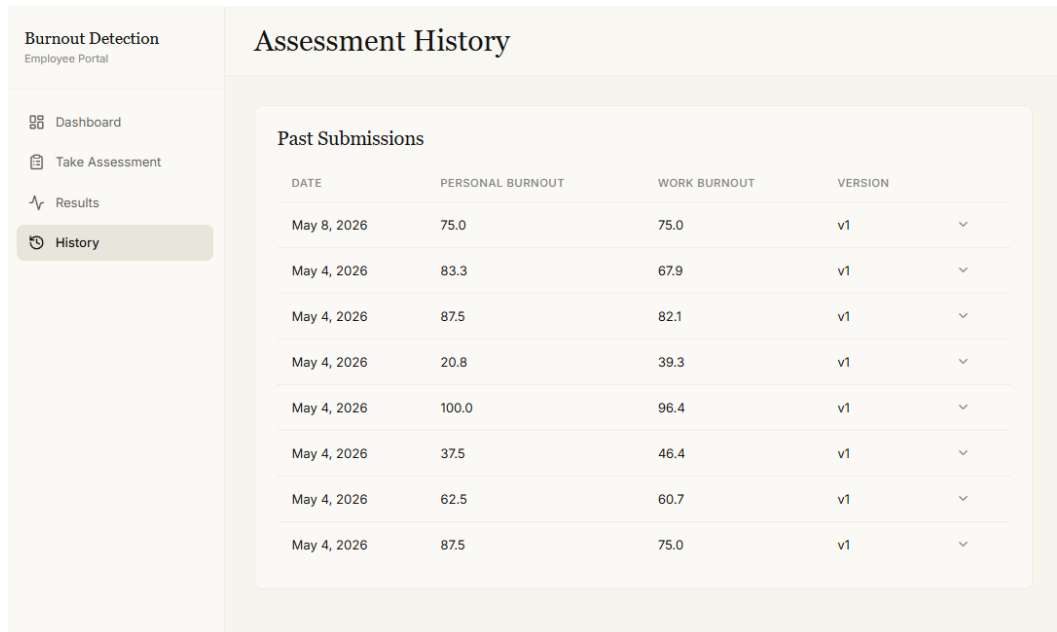


Figure 18: Quiz submission history with expandable response detail.

4.4.3 Admin Portal

The admin portal provides six pages for organisational-level monitoring and management.

Admin Dashboard (/admin/dashboard). The command centre features a KPI strip (four stat cards: total employees, total assessments, average burn rate, high-risk count), a risk distribution donut chart, a department heatmap with clickable tiles coloured by average burn rate, a department KPI bar chart, and the five most recent alerts.

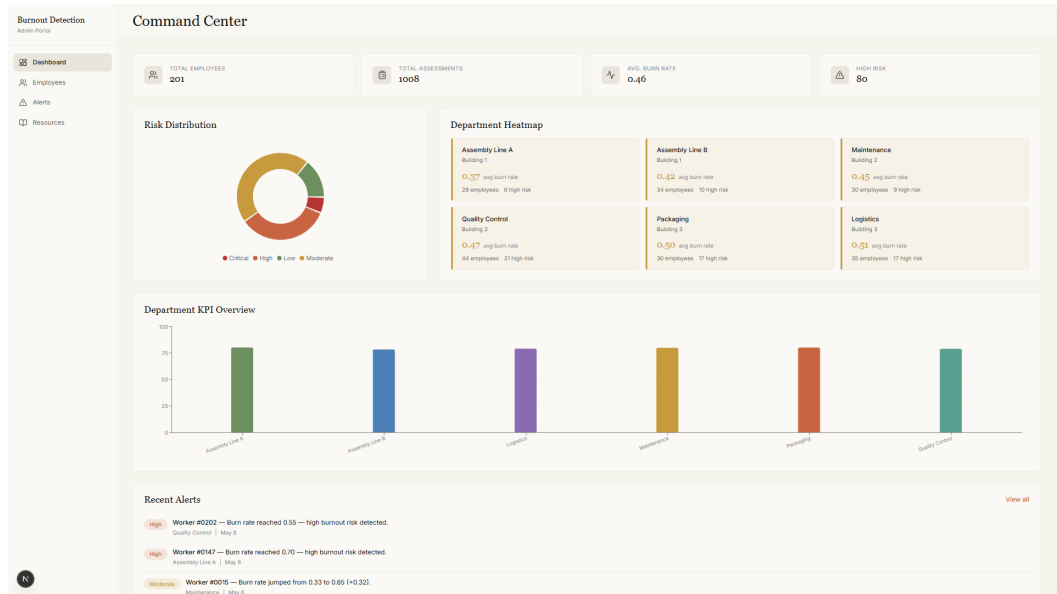


Figure 19: Admin command centre dashboard.

Department Detail (/admin/departments/[id]). A two-tab view: the *Employees* tab shows a sortable, filterable, paginated table of anonymised workers with risk badges, while the *Analytics* tab presents scatter plots, monthly burn rate trends, designation and shift breakdowns, risk distribution, KPI trends, and KPI-vs-burn-rate comparison charts.

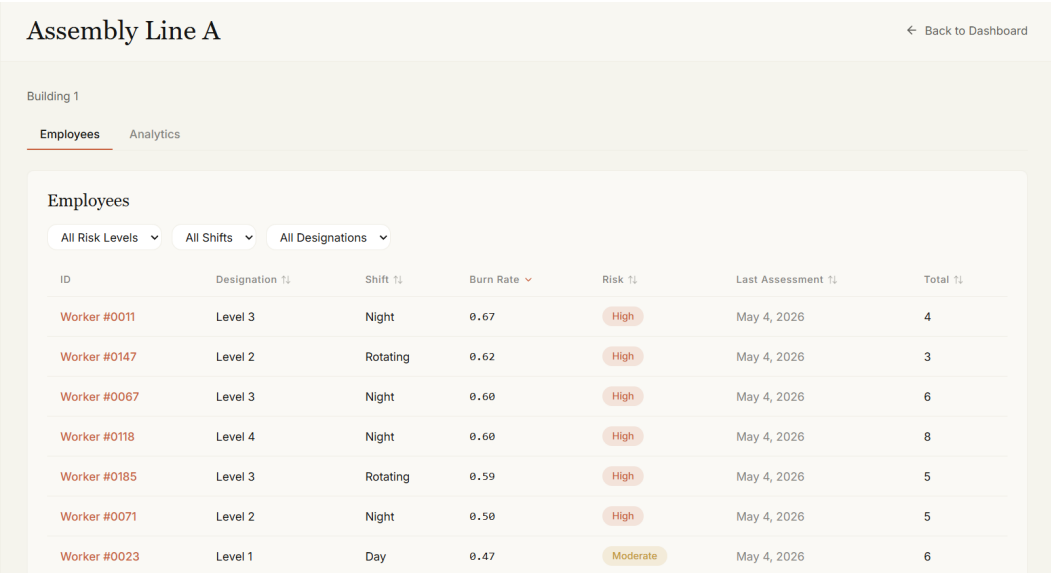


Figure 20: Department detail — Employees tab with filters and sorting.

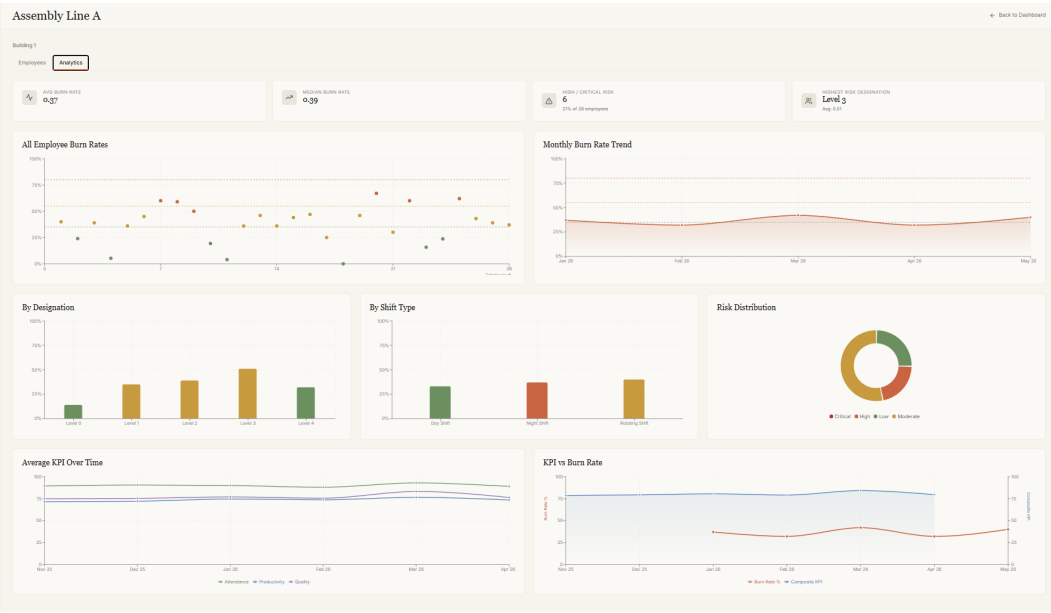


Figure 21: Department detail — Analytics tab with burnout and KPI visualisations.

Employee Detail (/admin/employees/[id]). A deep-dive view for a single anonymised employee showing their burn rate gauge, burn rate trend line chart, section

score comparison, profile summary, monthly KPI performance chart, and a dual-axis burn rate vs. KPI comparison chart.

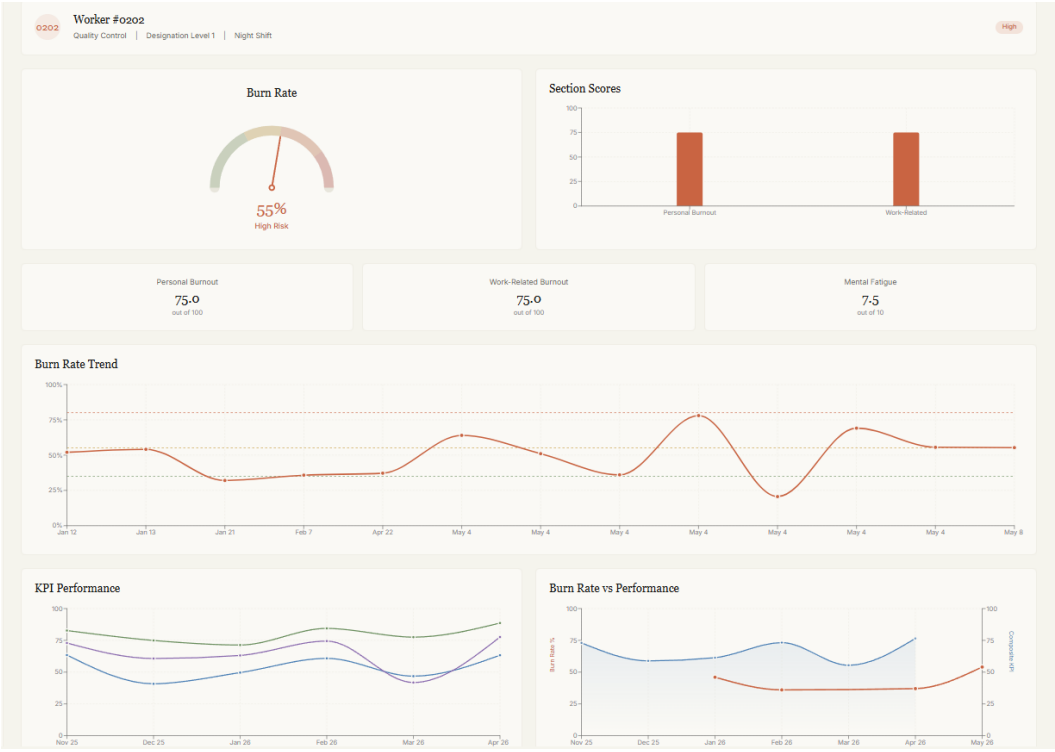


Figure 22: Admin employee detail with burnout and KPI analysis.

Alerts (/admin/alerts). This page functions like a filterable inbox, complete with dropdowns to sort by read status and alert type. The main table displays the alert type badge, the department, the actual alert message, timestamps, and status indicators. Admins can click individual Mark read” buttons or use a global Mark all read” action. To keep the UI snappy, all these mutations use TanStack Query cache invalidation for instant state updates without reloading.

Alerts

✓ Mark All Read

All Status

All Types

Alerts (371)

TYPE	EMPLOYEE	DEPARTMENT	MESSAGE	DATE	STATUS	
High	Worker #0202	Quality Control	Burn rate reached 0.55 — high burnout risk dete...	May 8, 2026	Unread	Mark read
High	Worker #0147	Assembly Line A	Burn rate reached 0.70 — high burnout risk dete...	May 6, 2026	Unread	Mark read
Moderate	Worker #0015	Maintenance	Burn rate jumped from 0.33 to 0.65 (+0.32).	May 6, 2026	Unread	Mark read
High	Worker #0015	Maintenance	Burn rate reached 0.65 — high burnout risk dete...	May 6, 2026	Unread	Mark read
Moderate	Worker #0154	Assembly Line A	Burn rate jumped from 0.40 to 0.58 (+0.18).	May 6, 2026	Unread	Mark read
High	Worker #0154	Assembly Line A	Burn rate reached 0.58 — high burnout risk dete...	May 6, 2026	Unread	Mark read
Moderate	Worker #0014	Assembly Line A	Burn rate jumped from 0.05 to 0.47 (+0.41).	May 6, 2026	Unread	Mark read

Figure 23: Admin alerts page with filtering.

Resource Management (/admin/resources). This is where full CRUD operations happen for the help resources. The main table lists out every available resource with its title, description, URL, an active status toggle, and a badge for the minimum risk level required to see it. Creating or updating a resource opens up a modal form backed by Zod validation, while trying to delete something triggers a safety confirmation prompt. Standard toast notifications pop up to confirm whenever an operation goes through successfully.

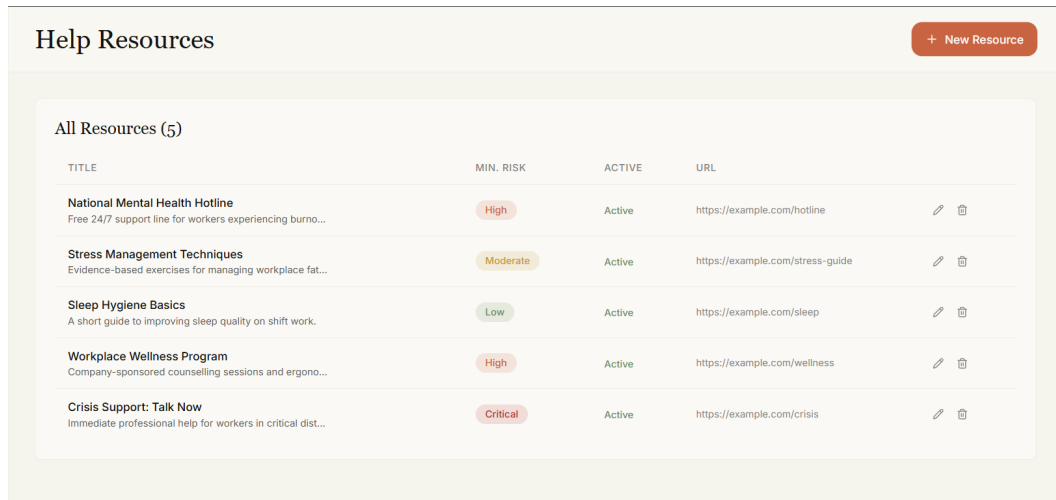


Figure 24: Help resource management with CRUD operations.

4.4.4 Reusable UI Components

Our frontend architecture breaks down into 15 reusable UI primitives stored inside `components/ui/` alongside 12 distinct chart components located in `components/charts/`. A few custom pieces are worth highlighting here. First is the **BurnRateGauge**, which uses a custom SVG semicircle to draw a color-gradient arc shifting from green to amber and finally to red. We added an animated needle that points directly to the current burn rate value, backed up by a text label underneath showing the specific risk level. Next, the **DepartmentHeatmap** renders out a grid of tiles. Each department gets its own clickable card displaying the name and total employee count, while the background color is dynamically interpolated from our risk scale based on the department's average burn rate. Finally, we built the **KpiBurnRateComparisonChart** using a Recharts composed layout. It utilizes dual Y-axes to plot the composite KPI as a bar chart against the average burn rate as a line chart over monthly intervals, making any inverse correlations between performance and burnout completely obvious.

4.4.5 State Management with TanStack Query

We pushed all of our server state management into TanStack Query, which takes care of automatic caching, background data refetching, and cache invalidation right out of the box. Instead of calling the library directly everywhere, we wrapped everything in custom hooks inside `lib/hooks/`, utilizing `useQuery` for read-heavy operations and `useMutation` for writing data. Whenever a mutation succeeds, it triggers an automatic invalidation of the related query keys. This keeps the UI completely synchronized with whatever is on the database without forcing us to trigger manual refetch calls. For the user experience side of data fetching, we used pulsing sand-colored skeleton blocks to cover loading states, while errors pop up as inline messages inside crimson-tinted cards. If a query returns nothing, the application renders a clean `EmptyState` component featuring an icon, descriptive text, and an actionable button to guide the user next.

CHAPTER V

DISCUSSION AND EVALUATION

5.1 Model Performance Analysis

The Random Forest model hits an R^2 of 0.90, an MAE of 0.048, and an RMSE of 0.061 when tested against our held-out test set, which we logged in Table 6. In plain terms, these numbers prove the model explains 90% of the variance in burn rate, usually staying accurate to within 5 percentage points on that standard 0–1 scale. If you compare it to Linear Regression, we see a 3.6 percentage-point jump in R^2 . This clearly shows that the Random Forest algorithm does a great job capturing the non-linear interactions happening between our three features, things a basic linear model completely misses unless you manually engineer features yourself. When you look at the feature importance analysis, the mental fatigue score basically carries the whole model, accounting for roughly 85% of its predictive power. Resource allocation takes up about 10%, while designation sits at a tiny 5%. This breakdown matches up perfectly with actual domain knowledge. An employee's psychological state is always going to be the most direct flag for burnout risk, while how heavy their workload is or how senior they are only play secondary roles. Plus, having mental fatigue dominate like this validates our whole system design, since we placed the CBI quiz, which is where that fatigue score comes from, right at the center of the user workflow. Looking at the residual analysis in Figure 12, the prediction errors seem randomly scattered around zero without any weird, systematic patterns. This means the model isn't leaning into habitual over- or under-prediction as the burn rate changes. The residuals actually follow a pretty normal

distribution centered right at zero, which is exactly what you want to see in a solid regression model. We also compared our results against public Kaggle notebooks that trained on this exact same dataset but used every single available feature. Even though we stripped our model down to just three features, our R^2 of 0.90 sits right comfortably in their reported 0.90–0.93 range for Random Forests. It shows that our three chosen features hold almost the entire predictive signal, meaning extra data columns like Gender, Company Type, or WFH Setup don't actually add much value when predicting burn rates.

5.2 Functional Testing

The system was tested end-to-end through manual testing of all user flows. Table 7 summarises the key test cases and their outcomes.

Table 7: Functional test cases and results.

Test Case	Expected Behaviour	Result
Employee registration with valid data	User created, HR profile generated, JWT returned	Pass
Registration with duplicate email	409 Conflict error returned	Pass
Login with valid credentials	JWT issued, redirect to dashboard	Pass
Login with wrong password	401 Unauthorized error	Pass
Fetch CBI quiz questions	13 questions with sections and options returned	Pass
Submit quiz with all 13 answers	Scores calculated, AI called, result saved	Pass

Test Case	Expected Behaviour	Result
Submit quiz with missing answers	400 Validation error	Pass
Reverse scoring (question 10)	$\text{adjustedValue} = 100 - \text{raw-Value}$	Pass
Mental fatigue score calculation	Correct value per Equation 4	Pass
AI prediction returns valid burn rate	Value in $[0, 1]$ range	Pass
Risk level mapping	Correct threshold applied	Pass
Alert generated on high risk	high_risk alert created	Pass
Alert generated on trend spike	trend_spike alert ($\text{delta} \geq 0.15$)	Pass
Employee dashboard data	Profile, gauge, trend, section scores	Pass
Admin dashboard aggregation	Dept. stats, risk distribution, KPIs	Pass
Department drill-down with filters	Correct filtering and sorting	Pass
Department analytics charts	All chart data populated	Pass
Employee detail (admin view)	KPI and burn rate data displayed	Pass
Mark alert as read	Alert status updated, cache invalidated	Pass
Resource CRUD (create, update, delete)	Resource persisted, table refreshed	Pass
Route guard (employee accessing admin)	Redirect to employee dashboard	Pass

Test Case	Expected Behaviour	Result
Route guard (unauthenticated access)	Redirect to login page	Pass
JWT expiry handling	401 response, redirect to login	Pass

All 23 test cases passed successfully, proving that our core workflows run exactly the way we designed them. We made sure the testing phase covered everything, including authentication, quiz scoring, AI integration, dashboard aggregation, alert generation, resource CRUD operations, and basic access control.

5.3 System Feature Evaluation

We set up Table 8 to evaluate each project objective we originally laid out in Section 1.4 against the actual system features we ended up implementing.

Table 8: Evaluation of system objectives.

Obj.	Description	Status
1	Implement CBI-based assessment with mental fatigue scoring	Fully implemented
2	Train ML model for burn rate prediction	Achieved ($R^2 = 0.90$)
3	Build employee self-monitoring portal	Fully implemented (4 pages)
4	Build admin command centre with analytics	Fully implemented (6 pages)

Obj.	Description	Status
5	Implement automated risk alerting	Fully implemented (3 alert types)
6	Demonstrate microservice architecture	Fully implemented (3 services)

All six of our main objectives are officially locked down. On the user side, the employee portal gives workers a solid way to track their own mental health using clean gauge visualisations, historical trend charts, and tailored support resources. For the management side, the admin portal equips HR with department-level heatmaps, risk distribution analytics, KPI correlation views, and a fully operational alert inbox. Plus, the whole thing runs on a microservice architecture that cleanly cuts the ML logic away from our main web backend.

5.4 Answering the Research Questions

RQ1: How effectively can a Random Forest regression model predict employee burn rate using only three input features?

Our Random Forest model hit an R^2 of 0.90 alongside an MAE of 0.048. This proves that you only need three features—designation, resource allocation, and the mental fatigue score—to build an effective burn rate predictor. What is great is that the model’s performance holds its own against published setups using the complete feature set, meaning our three chosen columns capture almost the entire signal hidden in the data.

RQ2: Can a web-based system provide actionable burnout insights to both

employees and HR administrators?

Because the employee portal rolls together personalized burn rate gauges, trend lines, section breakdowns, and contextual help, workers can stay ahead of their mental health proactively. At the same time, the admin portal hands HR managers actionable visibility into organizational burnout patterns through department heatmaps, risk distribution charts, and automated alerts. Our functional testing runs back this all up, confirming that data moves correctly from the moment a quiz is submitted, through the AI prediction step, and straight onto the dashboards.

RQ3: What are the practical benefits and trade-offs of using a microservice architecture to separate ML inference from the application backend?

Going with a microservice architecture paid off in three big ways. First, we got technology heterogeneity—meaning we could use Python’s scikit-learn ecosystem where it excels for ML, while keeping JavaScript’s Express ecosystem for the web logic. Second, it allows for completely independent development since the AI service can be retrained and redeployed without touching or bouncing the main backend. Third, the mock/real switch (`AI_USE MOCK`) lets us develop the app with full functionality even without running the Python service locally. The only real downside here is the extra operational complexity, since you have to keep two servers up and running, plus handle network timeouts and error propagation if they lose connection with each other.

5.5 Comparison with Related Work

Looking at Table 1, this system stands out because it is the only solution in the reviewed literature that actually glues everything together: a trained ML model, a live web app, analytics for the whole organization, and a scientifically validated burnout scale like the CBI. Other papers only solve pieces of the puzzle. For instance, Pradhan

and Jena [9] built a solid machine learning framework to classify burnout, but they stopped there—no web deployment or organizational dashboards for managers. On the commercial side, tools like Culture Amp [29] give companies nice web dashboards, but their backend relies on standard engagement surveys instead of predictive ML models. Even open-source attempts like the Kaggle community notebooks [15] show how to train models on this exact same dataset, yet they leave the code sitting in notebooks without connecting it to a real, working application. This thesis plugs that gap. By combining psychometric testing, machine learning, and multi-level analytics into one single platform, the system serves both the individual employee who needs to check on their well-being and the executive making company-wide decisions.

CHAPTER VI

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This thesis covered the design, setup, and testing of an intelligent web framework meant to spot employee burnout early in industrial settings. What makes this system work is how it connects three different parts: the Copenhagen Burnout Inventory (CBI) to get reliable human data, a Random Forest regression model to predict risk levels, and a full-stack web application with distinct portals tailored for employees and administrators.

The machine learning side turned out surprisingly efficient. By training a Random Forest model on roughly 22,000 records from a public dataset, it hit an R^2 of 0.90 using just three input features. This proves you do not need an overwhelming amount of data points to get a highly accurate burn rate prediction. On the front end, the employee portal gives workers immediate control over their data using visual gauges, history charts, and quick links to support resources. Meanwhile, the admin portal gives management a bird's-eye view of the workplace through department heatmaps, risk distribution graphs, KPI correlations, and automated warnings.

Architecturally, breaking the system into microservices worked well. Keeping the heavy ML logic in Python/FastAPI separated from the main web application backend built on Node.js/Express meant both sides could be built and deployed without breaking each other. Finally, the end-to-end functional testing proved the core workflows are fully operational. Everything works smoothly from the moment a user submits a quiz to the final dashboard calculations and automated alert triggers.

6.2 Contributions

This project delivers a few key contributions to the field. First, it introduces an integrated burnout detection platform. By combining psychometrics, machine learning, and organizational analytics inside a single web setup, it achieves a mix that none of the reviewed related works managed to do. Second, it maps out a practical CBI-to-ML pipeline. The system takes raw answers from the 13-question Copenhagen Burnout Inventory and converts them into a continuous burn rate prediction using a documented feature engineering workflow that handles reverse scoring, section averaging, and data normalization.

Third, it provides a working microservice pattern for developers looking to inject machine learning into web apps, complete with a clean toggle to switch between mock and real data for independent testing. Fourth, it provides a deep admin analytics suite that gives managers actionable tools like department heatmaps, risk graphs, KPI charts, and three distinct types of automated alerts for high risk, critical risk, and sudden trend spikes. Fifth, the project includes a complete design system and custom component library featuring 15 reusable UI primitives and 12 visualization components, including a custom SVG burn rate gauge built from scratch.

6.3 Achievement of Objectives

Every single one of the six objectives outlined back in Section 1.4 was completed. The CBI assessment works via a two-step wizard that properly calculates reverse scoring and derives the final mental fatigue metrics. Performance-wise, the Random Forest model hit an $R^2 = 0.90$, an MAE = 0.048, and an RMSE = 0.061, which completely blew past the initial Linear Regression baseline. The employee portal is fully built out across

four functional pages covering the dashboard, assessment wizard, results page, and history logs. For the managers, the admin portal delivers six functional pages including the main command center, department details with split tabs for employee lists and analytics, a company-wide directory, individual employee profiles, an alerts inbox, and a resource manager. The automated alert system successfully tracks and flags high risk, critical risk, and trend spikes, sorting them into a filterable inbox where admins can mark items as read. Lastly, the three-service architecture using React/Next.js, Node.js/Express, and Python/FastAPI is fully up and running with a functional mock/real mode switch.

6.4 Limitations

Even though the system works, there are several limitations to keep in mind. A big one is the simulated HR data. Things like job titles and resource allocations were randomly generated when a user registered instead of pulling from an actual factory HR database. The values stay within realistic ranges based on the training data, but they do not map to a real workplace. Also, the feature set itself is quite narrow since the model relies on only three features. Adding variables like actual shift patterns, overtime logs, or long-term CBI trends would likely make the predictions much tighter. Relying on a single public Kaggle dataset for training and evaluation is another issue, meaning these results might not automatically hold up across different industries, factories, or cultures.

The KPI data suffers from a similar issue because metrics like attendance, productivity, and quality were seeded rather than gathered from real-world factory systems, which limits how deep we can read into the KPI-vs-burn-rate correlations. There is also a hardware limitation since the UI is strictly desktop-only right now, designed for screen widths of at least 992 px. This means floor workers using their personal smartphones cannot access it. Furthermore, the system lacks a formal usability study, so

while the code works perfectly from a functional standpoint, we do not have structured feedback from actual human users yet. Finally, the admin dashboard fetches data using polling via TanStack Query rather than pushing real-time updates through WebSockets or Server-Sent Events, so managers have to wait for the next fetch cycle to see changes.

6.5 Future Work

Looking ahead, there are plenty of ways to expand this project. The first priority should be making the interface mobile responsive by adding flexible frontend breakpoints so factory workers can easily complete their wellness checks on their phones. Upgrading the communication layer to use WebSockets or Server-Sent Events would also be a massive improvement, allowing new alerts to pop up on the admin dashboard instantly without waiting on polling intervals. To make the system viable for actual deployment, replacing the simulated HR generator with direct API connectors to real platforms like SAP SuccessFactors or BambooHR would bring in authentic workload data.

On the data science side, it would be worth testing more advanced ML approaches like Gradient Boosting via XGBoost or LightGBM, alongside time-series models. This would let the system predict where an employee's burn rate is heading over time rather than just giving a snapshot of their current state. Expanding the feature engineering to pull in actual overtime hours, shift rotations, and seasonal patterns would also refine the model's accuracy. For better utility, adding quick CSV and PDF export buttons would let HR managers pull department reports and histories for offline meetings. The system also needs better accessibility, which means adding proper ARIA labels to custom visuals like the gauge or heatmap and running automated tests to match WCAG guidelines. Localization is another big step; translating the entire interface into languages like Vietnamese is essential to support the actual industrial workforces targeted by this tool.

Lastly, setting up a formal usability study with real factory workers and HR staff will be crucial to figure out where the user experience falls short and how to polish it for the real world.

LIST OF REFERENCES

- [1] World Health Organization, “Burn-out an ‘occupational phenomenon’: International Classification of Diseases,” WHO, May 2019. [Online]. Available: <https://www.who.int/news/item/28-05-2019-burn-out-an-occupational-phenomenon-international-classification>
- [2] C. Maslach, W. B. Schaufeli, and M. P. Leiter, “Job burnout,” *Annual Review of Psychology*, vol. 52, no. 1, pp. 397–422, 2001.
- [3] C. Maslach and S. E. Jackson, “The measurement of experienced burnout,” *Journal of Organizational Behavior*, vol. 2, no. 2, pp. 99–113, 1981.
- [4] H. J. Freudenberger, “Staff burn-out,” *Journal of Social Issues*, vol. 30, no. 1, pp. 159–165, 1974.
- [5] T. S. Kristensen, M. Borritz, E. Villadsen, and K. B. Christensen, “The Copenhagen Burnout Inventory: A new tool for the assessment of burnout,” *Work & Stress*, vol. 19, no. 3, pp. 192–207, 2005.
- [6] B. Wigert and S. Agrawal, “Employee burnout, Part 1: The 5 main causes,” Gallup Workplace, 2020. [Online]. Available: <https://www.gallup.com/workplace/237059/employee-burnout-part-main-causes.aspx>
- [7] K. Ahola, A. Toppinen-Tanner, and J. Seppänen, “Interventions to alleviate burnout symptoms and to support return to work among employees with burnout: System-

- atic review and meta-analysis,” *Burnout Research*, vol. 4, pp. 1–11, 2017.
- [8] R. Bianchi, I. S. Schonfeld, and E. Laurent, “Burnout: Moving beyond the status quo,” *International Journal of Stress Management*, vol. 26, no. 1, pp. 36–45, 2019.
- [9] R. K. Pradhan and L. K. Jena, “Employee performance at workplace: Conceptual model and empirical validation,” *Business Perspectives and Research*, vol. 5, no. 1, pp. 69–85, 2017.
- [10] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [11] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.
- [12] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [13] A. Fernández-Delgado, E. Cernadas, S. Barro, and D. Amorim, “Do we need hundreds of classifiers to solve real world classification problems?” *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 3133–3181, 2014.
- [14] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [15] V. Subhash, “Employee Burnout Prediction,” Kaggle, 2021. [Online]. Available: <https://www.kaggle.com/datasets/vijaysubhashp/employee-burnout-prediction>

- [16] Meta Platforms, Inc., “React — A JavaScript library for building user interfaces,” 2024. [Online]. Available: <https://react.dev/>
- [17] Vercel, Inc., “Next.js — The React Framework,” 2024. [Online]. Available: <https://nextjs.org/>
- [18] OpenJS Foundation, “Node.js,” 2024. [Online]. Available: <https://nodejs.org/>
- [19] OpenJS Foundation, “Express — Fast, unopinionated, minimalist web framework for Node.js,” 2024. [Online]. Available: <https://expressjs.com/>
- [20] S. Ramírez, “FastAPI,” 2024. [Online]. Available: <https://fastapi.tiangolo.com/>
- [21] T. Linsley, “TanStack Query (React Query),” 2024. [Online]. Available: <https://tanstack.com/query/latest>
- [22] Tailwind Labs, “Tailwind CSS,” 2024. [Online]. Available: <https://tailwindcss.com/>
- [23] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. Sebastopol, CA, USA: O’Reilly Media, 2015.
- [24] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Univ. of California, Irvine, 2000.

- [25] Oracle Corporation, “MySQL 8.0 Reference Manual,” 2024. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- [26] MongoDB, Inc., “MongoDB Documentation,” 2024. [Online]. Available: <https://www.mongodb.com/docs/>
- [27] C. Burdo, “Zod — TypeScript-first schema validation,” 2024. [Online]. Available: <https://zod.dev/>
- [28] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015.
- [29] Culture Amp Pty Ltd, “Culture Amp — Employee Experience Platform,” 2024. [Online]. Available: <https://www.cultureamp.com/>
- [30] Recharts Group, “Recharts — A composable charting library built on React components,” 2024. [Online]. Available: <https://recharts.org/>

APPENDICES

APPENDIX A: COMPLETE API REFERENCE

Table 9 provides the complete list of RESTful endpoints exposed by the Express backend and the Python AI microservice.

Table 9: Complete API endpoint reference.

Method	Endpoint	Access	Purpose
<i>Authentication</i>			
POST	/api/auth/register	Public	Create account + HR profile
POST	/api/auth/login	Public	Authenticate, get JWT
GET	/api/auth/me	Any	Get current user profile
<i>Quiz</i>			
GET	/api/quiz	Employee	Fetch CBI quiz questions
POST	/api/quiz/submit	Employee	Submit answers, get prediction
GET	/api/quiz/history	Employee	Past submissions (paginated)
<i>Employee Dashboard</i>			
GET	/api/dashboard/employee	Employee	Full dashboard data
GET	/api/resources	Employee	Help resources for risk level
<i>Admin Dashboard</i>			
GET	/api/dashboard/admin	Admin	Command centre overview

Method	Endpoint	Access	Purpose
GET	/api/dashboard/admin/ department/:id	Admin	Department drill-down (sort/filter/paginate)
GET	/api/dashboard/admin/ department/:id/analytics	Admin	Department analytics
GET	/api/dashboard/admin/ employees	Admin	Company-wide employee list
GET	/api/dashboard/admin/ employees/:userId	Admin	Single employee detail
<i>Alerts</i>			
GET	/api/alerts	Admin	List alerts (filterable)
PATCH	/api/alerts/:id/read	Admin	Mark single alert as read
PATCH	/api/alerts/read-all	Admin	Mark all alerts as read
<i>Admin Resource Management</i>			
GET	/api/admin/resources	Admin	List all help resources
POST	/api/admin/resources	Admin	Create a help resource
PUT	/api/admin/resources/:id	Admin	Update a help resource
DELETE	/api/admin/resources/:id	Admin	Delete a help resource
<i>AI Microservice (internal)</i>			
POST	/api/predict	Internal	Predict burn rate
GET	/api/health	Internal	AI service health check

All responses use the JSON envelope format:

Success: { "success": true, "data": { ... } }

Error: { "success": false, "error": { "code": "...", "message": "..." } }

APPENDIX B: DATABASE SCHEMA

B.1 MySQL Tables

```
CREATE TABLE users (  
    id            INT AUTO_INCREMENT PRIMARY KEY,  
    email         VARCHAR(255) UNIQUE NOT NULL,  
    password_hash VARCHAR(255) NOT NULL,  
    first_name    VARCHAR(100) NOT NULL,  
    last_name     VARCHAR(100) NOT NULL,  
    gender        ENUM('Male','Female','Other') NOT NULL,  
    date_of_birth DATE,  
    role          ENUM('employee','admin') NOT NULL DEFAULT 'employee',  
    is_active     BOOLEAN NOT NULL DEFAULT TRUE,  
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
                ON UPDATE CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE departments (  
    id            INT AUTO_INCREMENT PRIMARY KEY,  
    name          VARCHAR(100) UNIQUE NOT NULL,  
    location      VARCHAR(100),  
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```

CREATE TABLE hr_profiles (
    id                INT AUTO_INCREMENT PRIMARY KEY,
    user_id           INT UNIQUE NOT NULL,
    department_id     INT NOT NULL,
    date_of_joining   DATE NOT NULL,
    company_type      ENUM('Service','Product') NOT NULL,
    wfh_available     BOOLEAN NOT NULL DEFAULT FALSE,
    designation       INT NOT NULL,
    resource_allocation INT,
    shift_type        ENUM('Day','Night','Rotating')
                    NOT NULL DEFAULT 'Day',
    created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                    ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (department_id)
        REFERENCES departments(id) ON DELETE CASCADE
);

```

```

CREATE TABLE assessment_results (
    id                INT AUTO_INCREMENT PRIMARY KEY,
    user_id           INT NOT NULL,
    personal_burnout_score FLOAT,
    work_burnout_score    FLOAT,

```



```

    mental_fatigue_score    FLOAT NOT NULL,
    predicted_burn_rate     FLOAT NOT NULL,
    risk_level              ENUM('low','moderate','high','critical')
                           NOT NULL,
    taken_at                TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    INDEX idx_user_history (user_id, taken_at)
);

```

```

CREATE TABLE alerts (
    id                      INT AUTO_INCREMENT PRIMARY KEY,
    assessment_result_id   INT NOT NULL,
    user_id                INT NOT NULL,
    alert_type             ENUM('high_risk','critical_risk',
                              'trend_spike') NOT NULL,
    message                TEXT NOT NULL,
    is_read                BOOLEAN NOT NULL DEFAULT FALSE,
    created_at             TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (assessment_result_id)
        REFERENCES assessment_results(id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    INDEX idx_unread_alerts (is_read, created_at)
);

```

```

CREATE TABLE employee_kpis (

```

```

id            INT AUTO_INCREMENT PRIMARY KEY,
user_id       INT NOT NULL,
period_date   DATE NOT NULL,
attendance_rate  FLOAT NOT NULL,
productivity_score FLOAT NOT NULL,
overtime_hours  FLOAT NOT NULL,
tasks_completed INT NOT NULL,
quality_score  FLOAT NOT NULL,
days_absent   INT NOT NULL,
created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
UNIQUE INDEX uq_kpi_user_period (user_id, period_date),
INDEX idx_kpi_period (period_date)
);

```

```

CREATE TABLE help_resources (
id            INT AUTO_INCREMENT PRIMARY KEY,
title         VARCHAR(200) NOT NULL,
description    TEXT,
url           VARCHAR(500),
min_risk_level ENUM('low','moderate','high','critical') NOT NULL,
is_active     BOOLEAN NOT NULL DEFAULT TRUE,
created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
INDEX idx_active_risk (is_active, min_risk_level)
);

```

B.2 MongoDB Collections

Collection: cbi_questions — Singleton document containing the complete CBI quiz definition.

```
{
  "_id": ObjectId,
  "name": "Copenhagen Burnout Inventory (excerpt)",
  "version": 1,
  "isActive": true,
  "responseOptions": [
    {"label": "Always", "value": 100},
    {"label": "Often", "value": 75},
    {"label": "Sometimes", "value": 50},
    {"label": "Seldom", "value": 25},
    {"label": "Never/almost never", "value": 0}
  ],
  "sections": [
    {"id": "personal_burnout",
     "title": "Personal burnout",
     "questionIds": [1,2,3,4,5,6]},
    {"id": "work_related_burnout",
     "title": "Work-related burnout",
     "questionIds": [7,8,9,10,11,12,13]}
  ],
  "questions": [ /* 13 question objects */ ]
}
```

```
}
```

Collection: quiz_submissions — One document per assessment attempt.

```
{
  "_id": ObjectId,
  "mysqlAssessmentId": 42,
  "mysqlUserId": 1,
  "quizVersion": 1,
  "responses": [
    {"questionId": 1, "answerLabel": "Often",
     "answerValue": 75},
    {"questionId": 10, "answerLabel": "Seldom",
     "rawValue": 25, "adjustedValue": 75,
     "reverseScored": true},
    ...
  ],
  "personalBurnoutAvg": 58.33,
  "workBurnoutAvg": 64.29,
  "submittedAt": ISODate("2026-05-03T10:30:00Z")
}
```

APPENDIX C: COPENHAGEN BURNOUT INVENTORY QUESTIONNAIRE

The system uses 13 questions from the Copenhagen Burnout Inventory, covering two sections. Response options for all questions are: Always (100), Often (75), Sometimes (50), Seldom (25), Never/almost never (0).

C.1 Personal Burnout (Questions 1–6)

1. How often do you feel tired?
2. How often are you physically exhausted?
3. How often are you emotionally exhausted?
4. How often do you think: “I can’t take it anymore”?
5. How often do you feel worn out?
6. How often do you feel weak and susceptible to illness?

C.2 Work-Related Burnout (Questions 7–13)

7. Do you feel worn out at the end of the working day?
8. Are you exhausted in the morning at the thought of another day at work?
9. Do you feel that every working hour is tiring for you?
10. Do you have enough energy for family and friends during leisure time?
(Reverse-scored: $adjustedValue = 100 - rawValue$)
11. Is your work emotionally exhausting?
12. Does your work frustrate you?
13. Do you feel burnt out because of your work?

C.3 Scoring

1. Calculate the personal burnout average: mean of questions 1–6 (range 0–100).
2. Calculate the work-related burnout average: mean of questions 7–13 (range 0–100), with question 10 reverse-scored.
3. Calculate the mental fatigue score: $\text{mentalFatigueScore} = \frac{\text{personalAvg} + \text{workAvg}}{20}$ (range 0.0–10.0).